in28minutes

Getting Started with Azure AI-901

Hello and Welcome to in28minutes

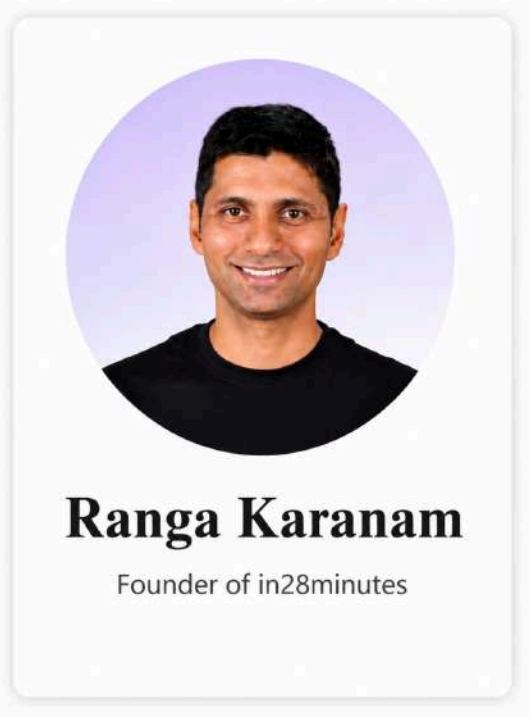
Who We Are: in28minutes, founded by Ranga Karanam

Our Reach: Millions of learners taught worldwide

- But our focus is not the numbers
- **Our focus is your progress**

Our Style: Practical. Story-driven. Step-by-step.

Our Goal: Help you move from where you are to where you want to be



Getting Started with AI-901

Evolving Ecosystem: AI capabilities in Azure are constantly evolving

Exam Focus: Two key areas

- **Conceptual Knowledge**: Understand core AI concepts and Azure AI capabilities
- **Foundational Technical Skills**: Understand how to configure and use common Azure AI solutions

Course Design: Learn concepts first, then reinforce them with practical examples

Course Goal: Help you understand Azure AI, apply the concepts, and get certified with confidence

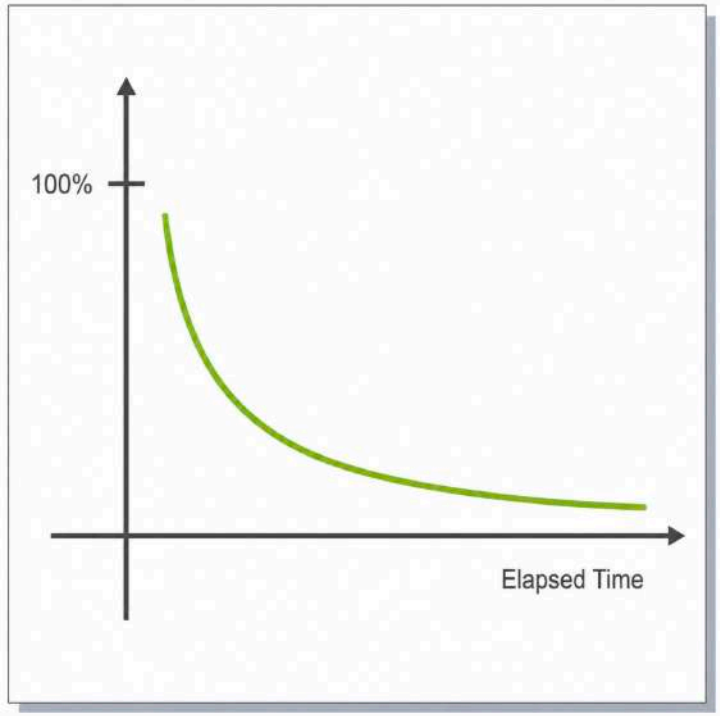
How Do You Put Your Best Foot Forward?

Challenging Certification: You are expected to understand and remember many services

Human Memory Fades: We naturally forget things over time

How Can You Remember More?

- **Active Learning**: Think, practice, and take notes as you learn
- **Regular Review**: Revisit presentations periodically to reinforce your memory



Our Approach to a Great Learning Experience

Three-Pronged Approach to reinforce concepts:

- **Presentations**: Build conceptual understanding
- **Demos**: See concepts in action
- **Quizzes**: Reinforce and test your learning
  - Text quizzes
  - Video quizzes

Recommended:

- Take your time and replay videos whenever needed
- Have fun while learning!

What Learners Are Saying

in28minutes: Trusted by millions of learners worldwide

What Learners Say:

- ⭐⭐⭐⭐⭐ - "Excellent course. Absolutely loved it. Instructor explained it very well."
- ⭐⭐⭐⭐⭐ - "Excellent course. I recommend it to anyone looking to build a solid foundation in AI fundamentals!"
- ⭐⭐⭐⭐⭐ - "This is a great course. I used it to prepare for the exam and passed today with a score of 911"

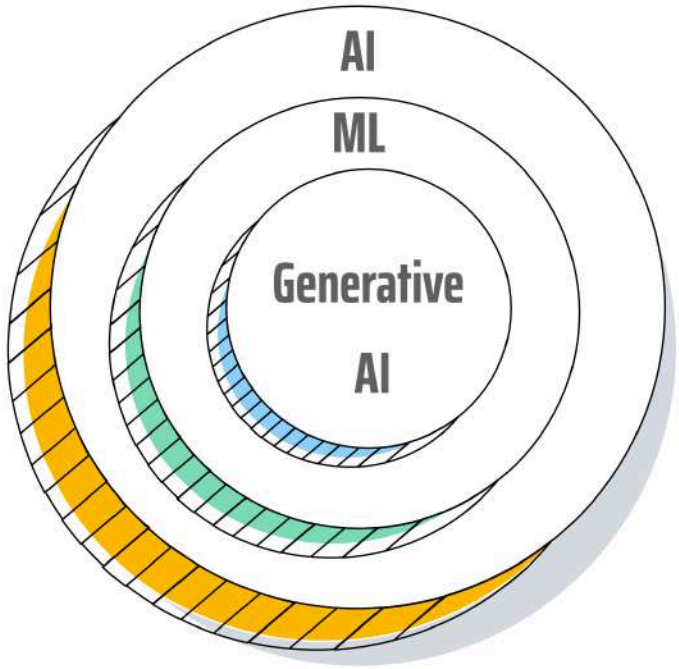
in28minutes

Getting Started with Artificial Intelligence

Section Overview - Introduction to AI, ML and Generative AI

Focus: What will we learn?

- What is Artificial Intelligence?
- What is Machine Learning?
- How does Machine Learning work?
- Machine Learning vs Traditional Programming
- What is Generative AI?
- How is Generative AI different from Traditional AI and ML?

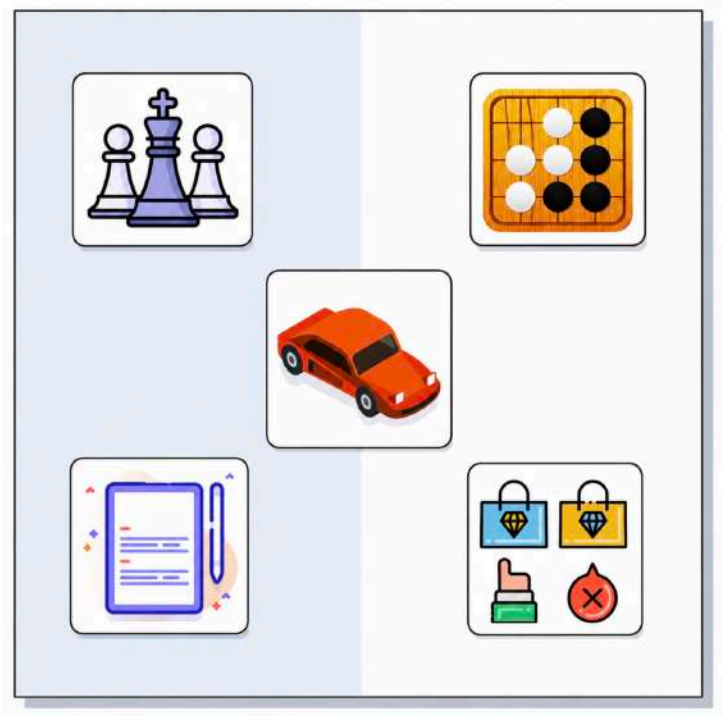


Foundation: Build the core concepts needed for the rest of the course

What is Artificial Intelligence?

Humans are great at:

- **Decision-Making**: Choose from multiple options
- **Visual Perception**: Recognize faces and objects
- **Language Understanding**: Understand written text
- **Speech Recognition**: Understand spoken language
- And a lot more..



Artificial Intelligence: Theory & development of computer systems able to perform tasks normally requiring human intelligence

- **Simplified Goal**: Make computers do things that normally require human intelligence

Are All AI Systems Equally Intelligent?

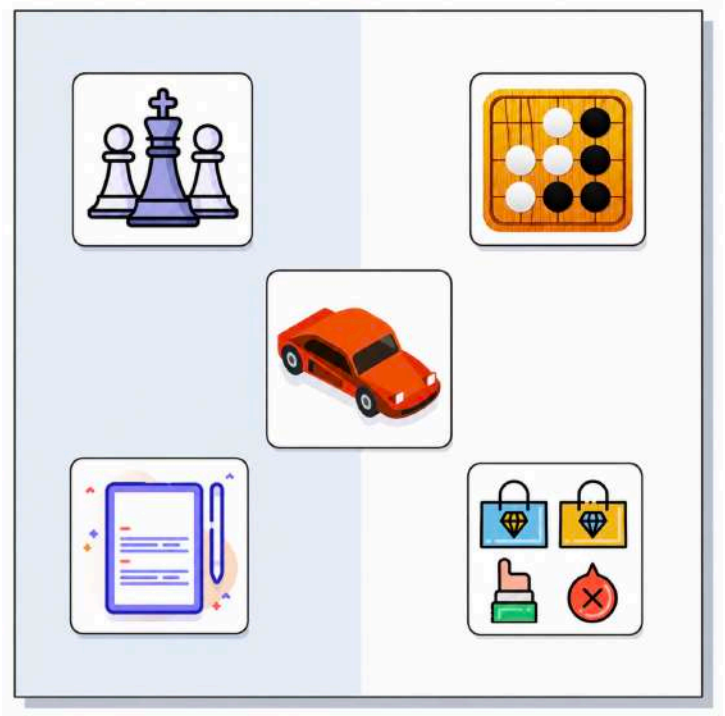
AI can have different levels of capability

Some AI systems are designed to:

- Perform **one specific task**
- Perform **many different tasks**

This leads to two broad types of AI:

- **Strong AI**
- **Narrow AI**



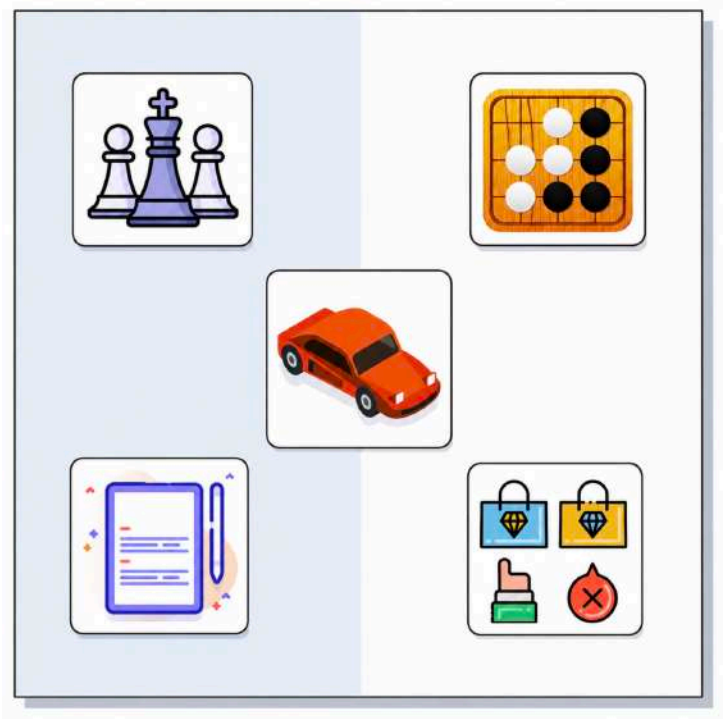
Understanding Types of AI - Strong AI

Strong AI: Goal is to create machines that are as intelligent as humans

Capabilities: Solve problems, learn, and plan for the future

- Expert at everything
- Make complex decisions
- Learns like a child

Also Known As: General AI or Artificial General Intelligence (AGI)



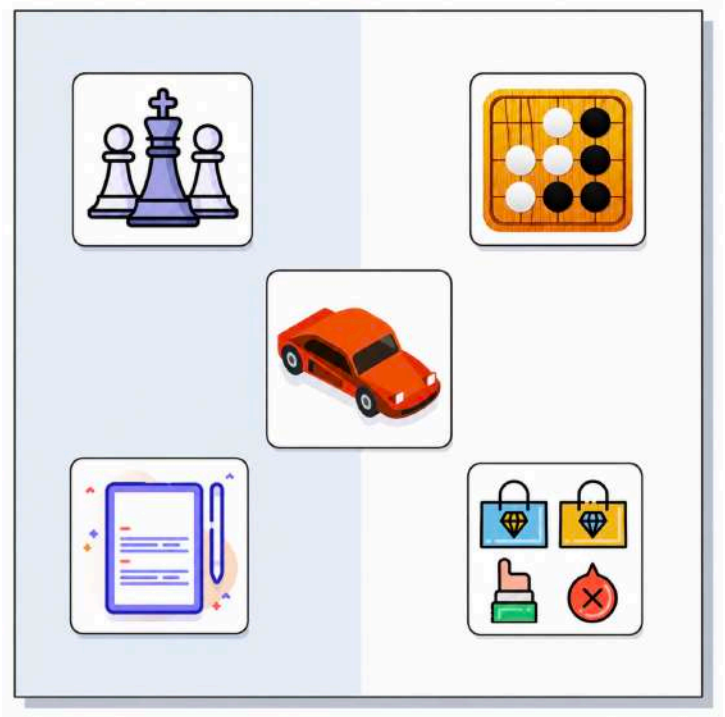
Understanding Types of AI - Narrow AI

Narrow AI: AI focused on a specific task

Examples:

- **Self-driving Car**: Can drive but cannot play chess
- **Chess App**: Can beat champions but cannot drive
- **Weather Predictor**: Knows weather but cannot write poetry

Also Known As: Weak AI



Getting Started with AI - Scenarios

Scenario	Solution
What is the Goal of AI?	Make computers do things that normally require human intelligence
Strong or Narrow AI: Build a system that can learn any game, write poetry, and drive a car like a human	Strong AI (General AI)
Build a computer system that focuses on a specific task such as self-driving, virtual assistants, or object detection	Narrow AI (or Weak AI)
The AI used in your smartphone to recognize your face	Narrow AI

Why are we talking a lot more about AI now?

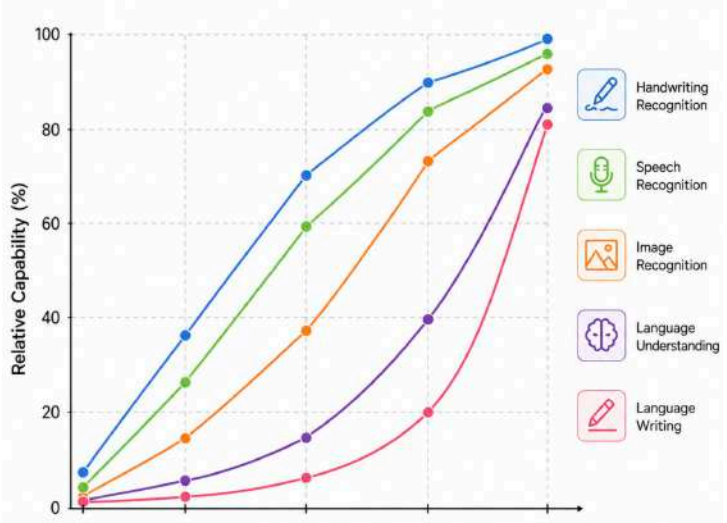
AI is Not New: Researchers have been working on AI since 1950s

Question: Why are we talking a lot more about AI now?

- **Answer**: AI capabilities improved dramatically in recent decades

Question: How can we measure progress in Artificial Intelligence?

- **Early Answer**: The **Turing Test**



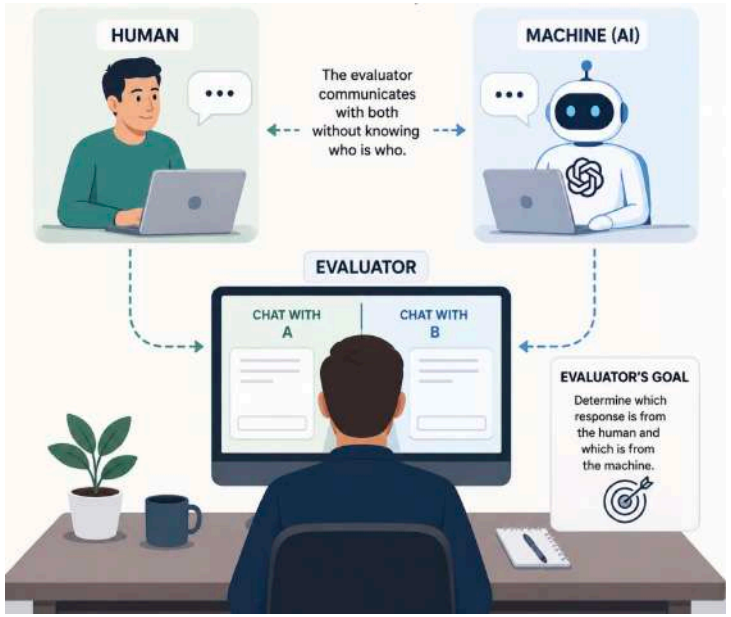
What is Turing Test?

Goal: Check if a AI system can communicate like a human

- **1**: An evaluator chats with a human and a AI system
- **2**: The evaluator asks questions through text
- **3**: Both respond to questions
- **4**: Evaluator tries to identify the machine

Early AI: AI systems struggled to imitate human conversation

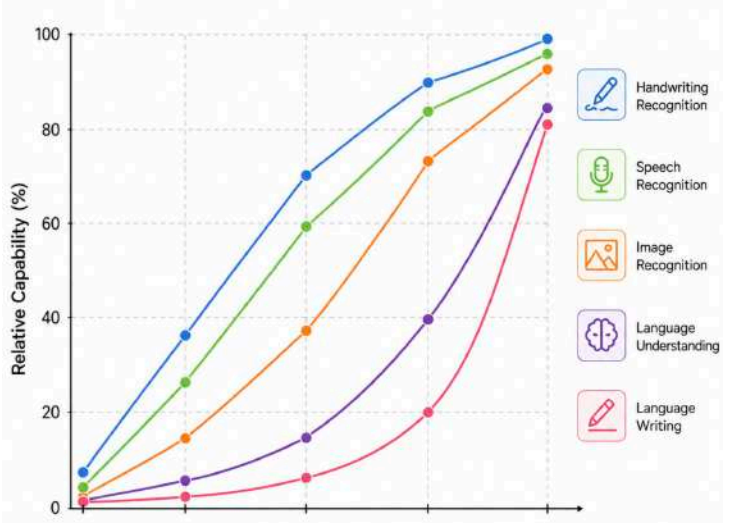
Today: Advanced AI systems can often produce human-like responses



AI Has Made a Lot of Progress

AI Improved Rapidly: Across many areas

- **Image Recognition**: Recognizing object in images
- **Speech Recognition**: Convert spoken words into written text
- **Language Generation**: Large language models are capable of fluent writing
- And a number of others



How: How did this progress happen?

- Coming up!

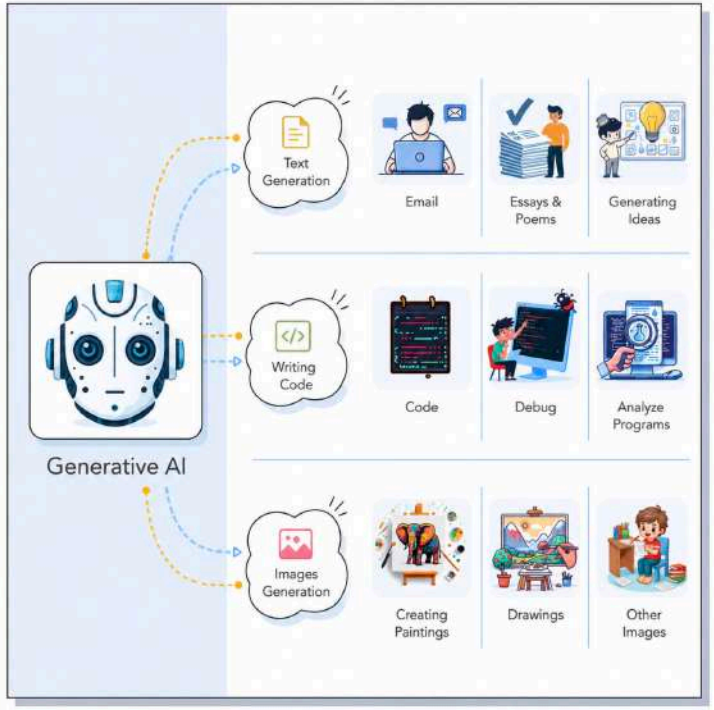
Playing with Generative AI

Checklist: Generate a list of items I need for a 15-day Everest Base Camp trek

- Update the list assuming I will stay at a tea house as part of a guided group

Multimodal: Let's create an image showing the atmosphere at a cricket match

- A vibrant, high-energy night scene at an Indian Premier League (IPL) cricket match. Focus on the electric atmosphere. Show fans wearing team jerseys, waving flags, and cheering loudly. Include digital scoreboards and a sense of celebration.

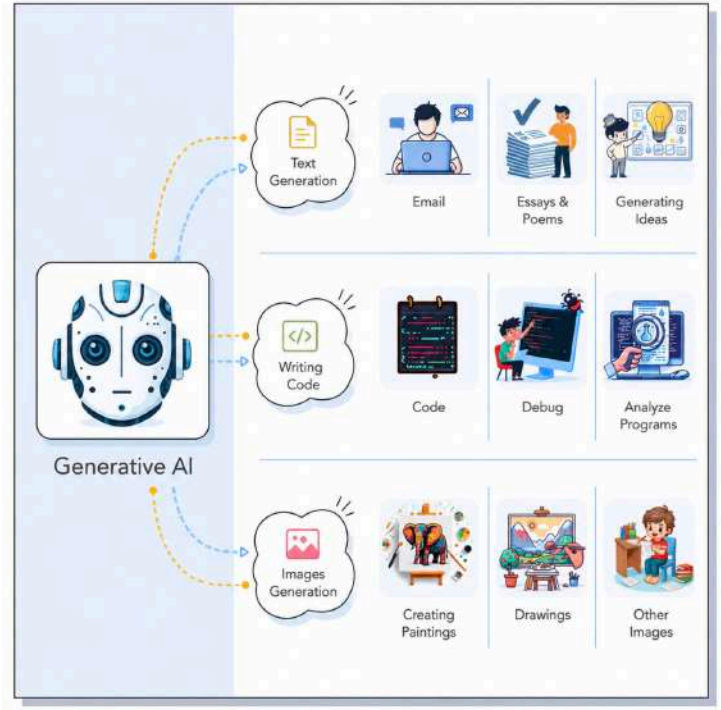


Playing with Generative AI - Observations

Powerful, Not Perfect: Can produce incorrect, biased, or inappropriate responses

- **Use with Judgment**: Outputs should be reviewed and verified
- **Still Valuable**: When used with the right expectations, Generative AI can be extremely useful

How does it work?: Let's start with Machine Learning and build our way to Generative AI



in28minutes

Getting Started with Machine Learning

How is Traditional Programming Done? [1/2]

Approach: Rule-based logic

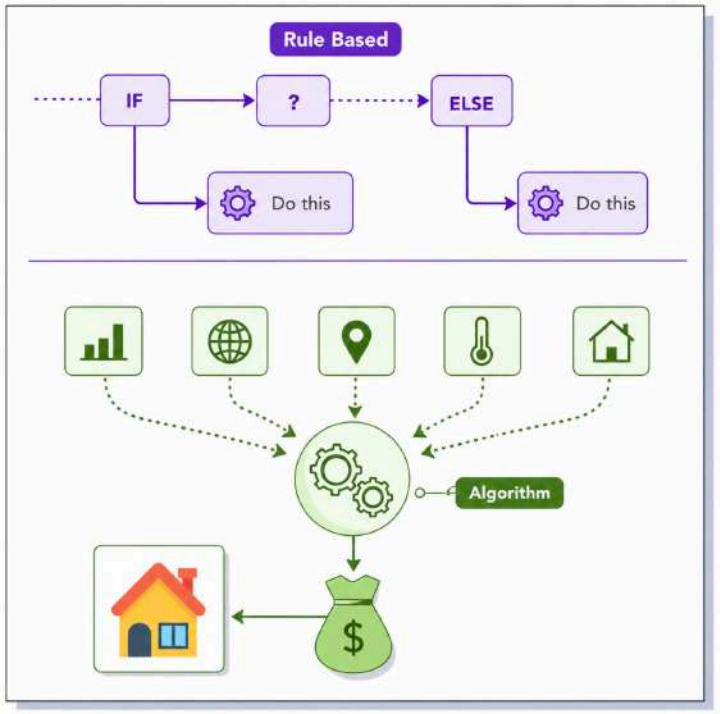
Mechanism: You explicitly tell the computer what to do

Flow:

- **Input**: Data
- **Program**: Rules such as IF this DO that
- **Output**: Result

Example: Calculating tax

- **Rule**: If income > 50k, tax is 20%



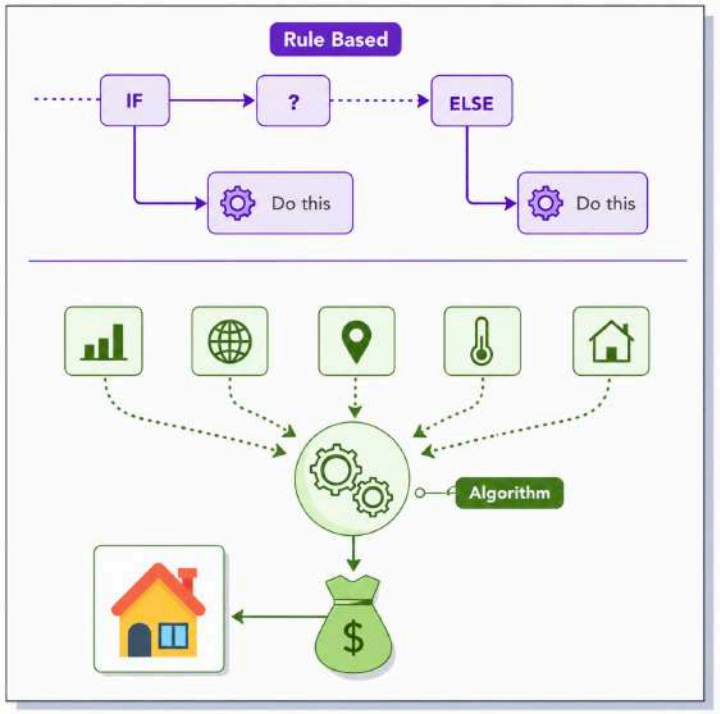
How is Traditional Programming Done? [2/2]

Example: Predicting house price

Approach: You design an algorithm that considers all key factors

- Location
- Home size
- Age
- Condition
- Market

Result: The computer does exactly what you tell it, nothing more and nothing less



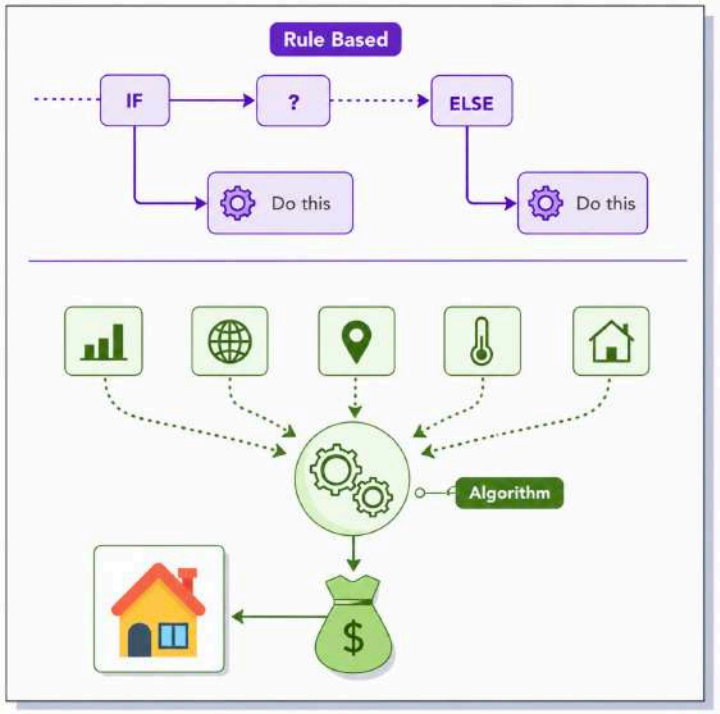
What is the Limitation of Traditional Programming?

Traditional Programming: Input --> Program (Rules or Algorithm) --> Output

Think About This: Can you write the rules to recognize a cat?

- **Example**: If pixel 1 is light and pixel 2 is dark, and so on
- **Reality**: Almost impossible to hand-code such kind of rules

Result: We hit a wall with traditional coding



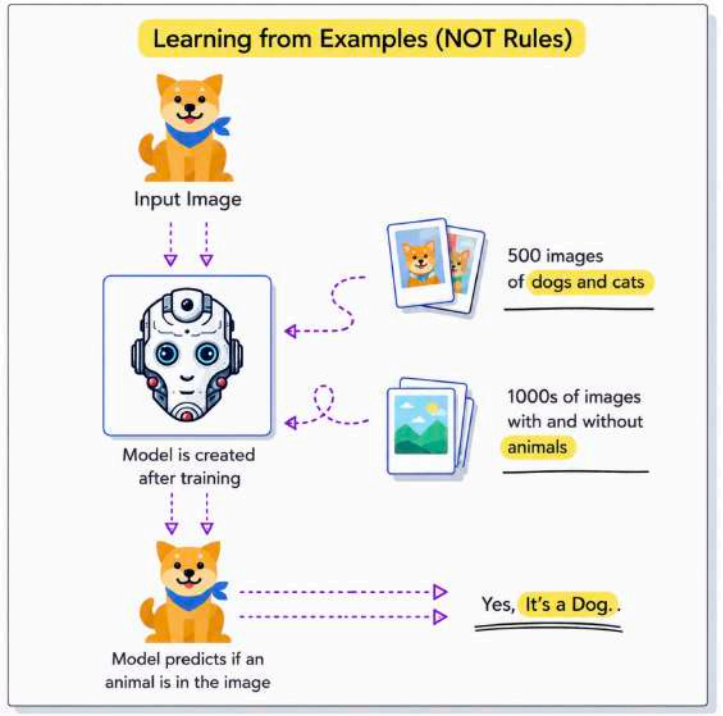
How Does Machine Learning Work?

What Happens?: ML learns from data

Machine Learning: Give examples, ML finds patterns, then predicts the outcome

Example:

- **House Price Prediction**: Learn from past sales data



Machine Learning - Simplified Workflow

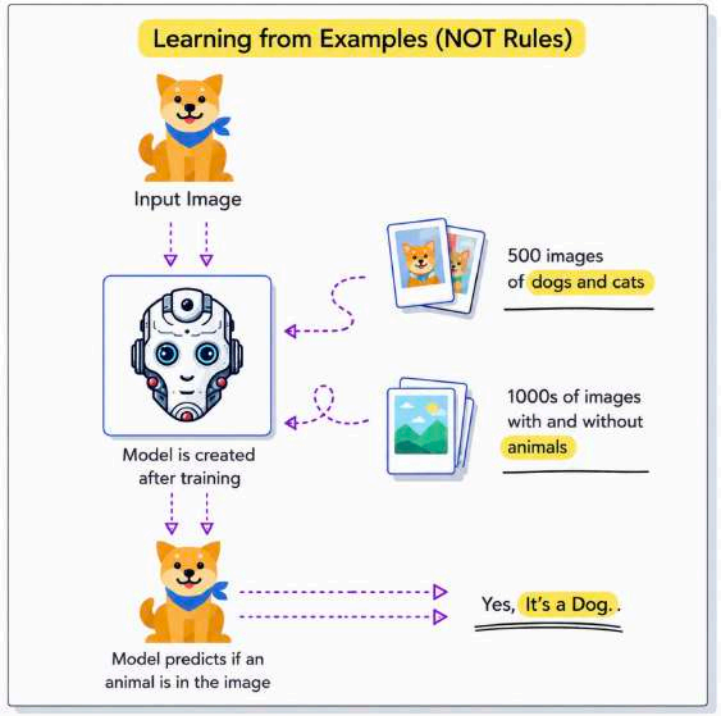
Step 1 - Obtain Data: Provide examples

- **Example**: Photos of cats, dogs and other animals with labels

Step 2 - Train Model: The system analyzes images to find patterns

- **Learned Patterns**: Shapes, edges, and textures

Step 3 - Predict: Give it a new photo and get a prediction



Traditional Programming vs Machine Learning

Traditional Programming	Machine Learning
Programmer writes the rules (program)	Model learns the rules from data
Input → Program (Rules or Algorithm) → Output	Input + Expected Outputs → Training → Model
Works well when rules are known	Works well when rules are difficult to define
Example: Tax calculation	Example: House price prediction
Example: Payroll processing	Example: Spam email detection

Machine Learning Fundamentals - Scenarios

Scenario	Solution
Developer writes logic to tell the computer what to do	Traditional Programming
System finds patterns from data that humans cannot explicitly define	Machine Learning
Category of AI that focuses on learning from data	Machine Learning
How is ML different from traditional programming?	Traditional Programming uses rules, ML uses examples

in28minutes

Getting Started with Machine Learning Stages

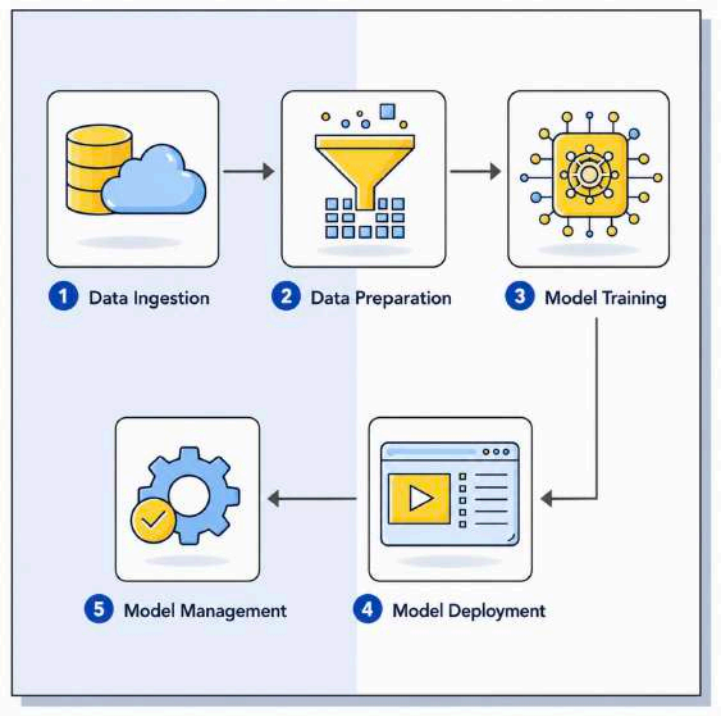
ML Lifecycle: Data Related Stages

Data Ingestion: Collect raw data from different sources

- **Example**: Streaming user activity from a mobile app
- **Example**: Pulling transaction logs from a sales system

Data Preparation: Clean and format data for training

- **Example**: Remove duplicates
- **Example**: Handle missing values in patient health data



ML Lifecycle: Model Related Stages

Model Training: Use data to train the model

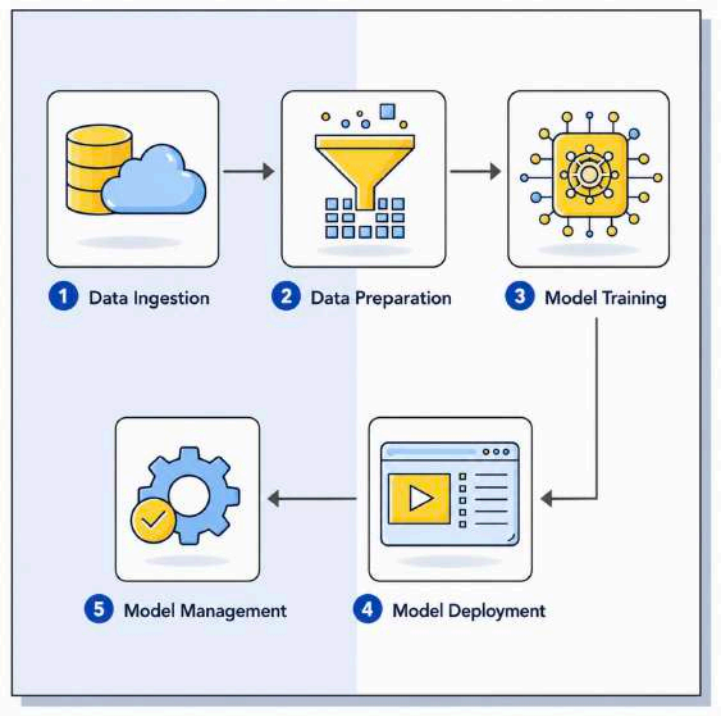
- **Example**: Train a fraud detection model with transactions
- **Example**: Train a price prediction model for used cars

Model Deployment: Make the trained model available to users

- **Example**: Deploy a chatbot in production (Inference)

Model Management: Monitor and maintain the model after deployment

- **Example**: Update the model as new financial data comes in (Re-Training)



ML Lifecycle Recap: End-to-End Example

Use Case: ML model to detect spam in email

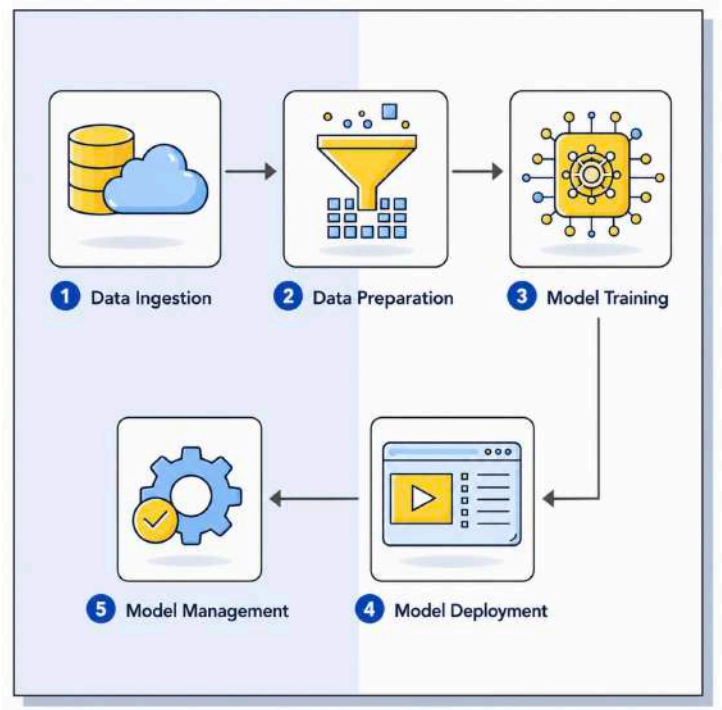
1: Collect email history

2: Prepare labels

3: Train with examples

4: Deploy into email platform

5: Manage performance



Identify the Stage in ML Lifecycle - 1

Activity	Stage
Pulling product transaction logs from a retail store's log store	Data Ingestion
Streaming real-time user activity from a mobile app	Data Ingestion
Removing duplicate customer records from the training dataset	Data Preparation
Filling in missing values in patient data using averages	Data Preparation
Labeling emails as spam or not spam for training	Data Preparation
Using thousands of labeled housing records to teach a model to predict price	Model Training
Training a model to detect credit card fraud using past transaction data	Model Training

Identify the Stage in ML Lifecycle - 2

Activity	Stage
Making your trained model available via an API to a web application	Model Deployment
Deploying an image classification model into a mobile app	Model Deployment
Monitoring how accurate your chatbot is after deployment	Model Management
Updating your model with recent data after noticing performance drop	Model Management
Creating a dashboard that displays model inference accuracy trends	Model Management

in28minutes

Getting Started with Generative AI

Generative AI - Generates New Content

Goal: Generate New Content

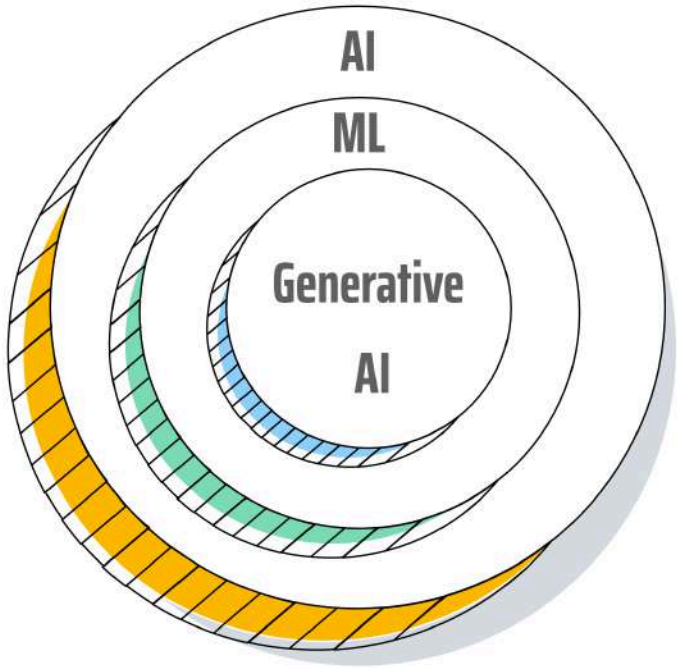
- Instead of making predictions, Generative AI focuses on creating new content

Text Generation: Writing e-mails, essays & poems. Generating ideas.

Code Generation: Generate code snippets. Create unit tests.

Media Creation: Design images. Create instructional videos.

And a lot more features!

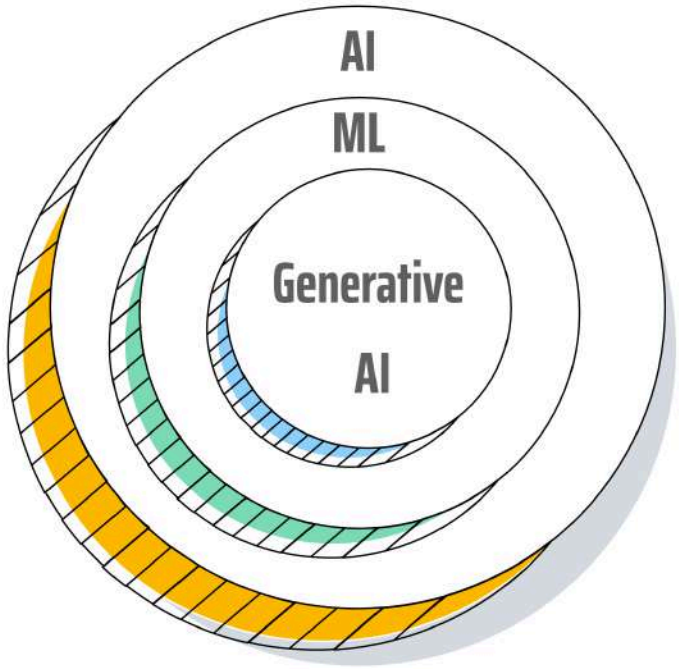


Generative AI - How is it different?

Artificial intelligence (AI): Create machines that simulate human-like intelligence and behavior

Machine learning (ML): A subset of AI where machines learn from data

Generative AI: AI that learns patterns from data to **generate new content**



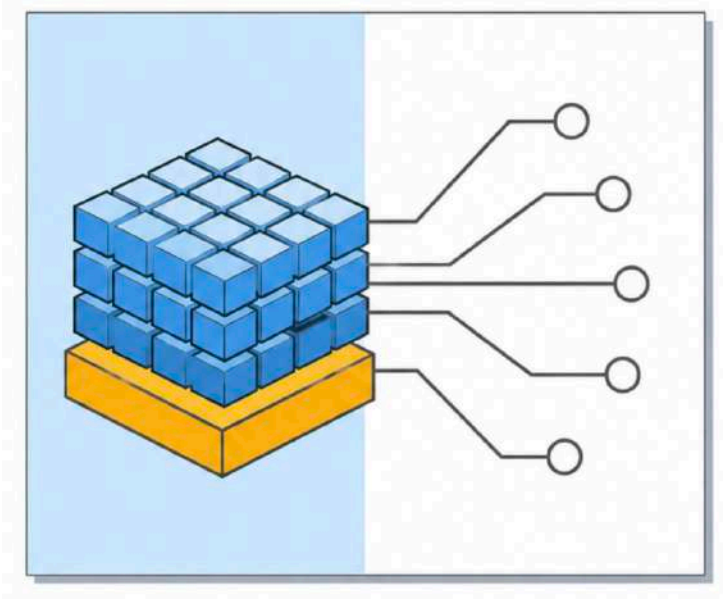
Foundation of Generative AI - The Foundation Models

Foundation Model: The core model powering many generative AI applications

- **One Model, Many Tasks**: Answer questions, summarize, generate code, create content
- **Examples**: GPT, Gemini, Claude, Llama

Pre-trained: Trained on massive amounts of text, code, images, and other data

- **Expensive to Train**: Requires significant compute, time, and data
- **Easy to Reuse**: Build applications without training a model from scratch

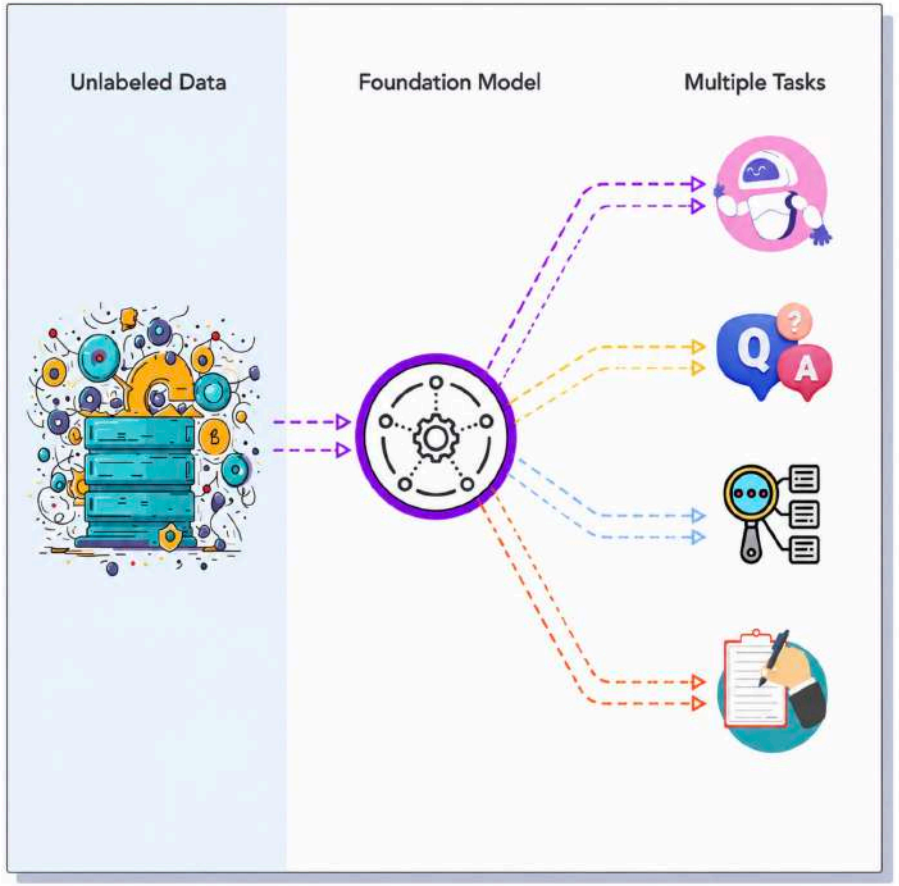


Training Foundation Models is Complex

Complex Training Process: Training is computationally intensive and complex

- **Massive Datasets**: Books, code, web pages, images, captions, and more
- **Specialized Hardware**: GPUs, TPUs, distributed storage, and parallel processing
- **Takes Time**: Training runs for weeks or months

Outcome: Broad general knowledge - understands how humans write, speak and code



Foundation Models Are Multi Task Models

Traditional ML Models:

- Needed task specific training
- Multiple tasks => Multiple trainings => Multiple models

Foundation Models: Multi Task Models

- Trained once (called pre-training)
- Same model can be used for multiple tasks
  - Chatbot, Classification, Summarization, etc.
- Some models are multi modal as well: Text, video, audio, image...

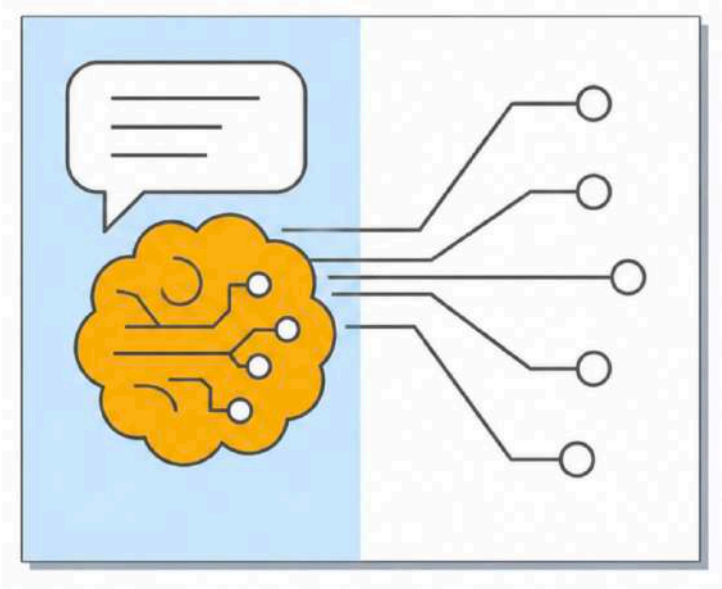
Text Foundation Models are called Large Language Models

Trained on **huge volumes** of diverse text data

- Books, articles, websites, code, and more

Learn to **predict the next token** in a sequence

- Example: "The cat sat on the ..."
- Model gives probabilities:
  - "mat" (40%), "table" (20%), "chair" (20%), "moon" (10%)
- It usually picks the most likely token — but we can tune settings to make it more creative
  - e.g., using the temperature parameter



Usecases: Chat, summarization, translation, Q&A, ..

Diffusion Models: Turning Noise into Art

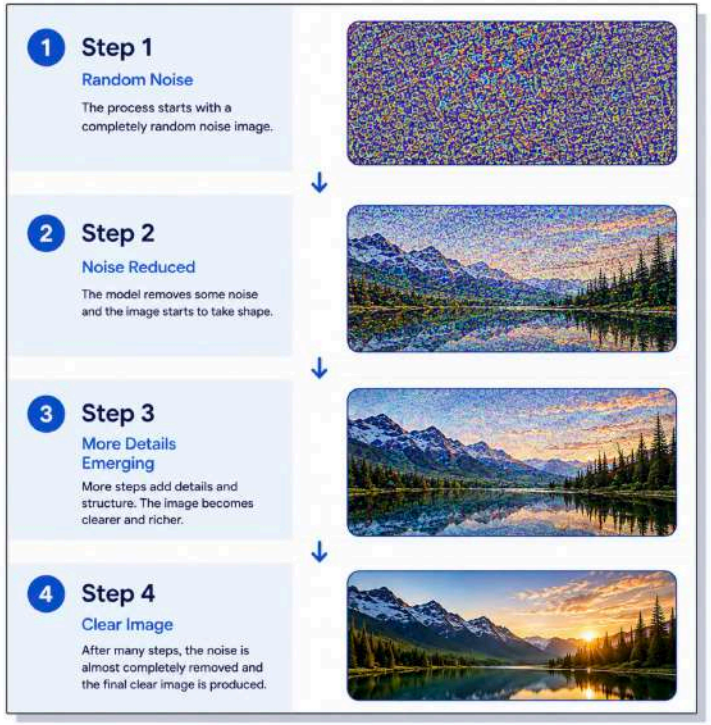
Diffusion models: Great at generating images, audio, and video

How they work:

- Start with **random noise**
- Iteratively **refine it** step-by-step
- Result: **Structured, realistic output**

Used for:

- Creating AI art and realistic pictures from text prompts
- Generating life-like human voices or music
- Producing short AI-generated videos



Foundation Models - Scenarios

Scenario	Solution
Models trained on large, diverse datasets across domains. Adaptable to a wide variety of tasks.	Foundation Models
Text foundation models trained on huge volumes of diverse text data.	Large Language Models
Models that generate images, audio, or video starting with random noise and refining step-by-step	Diffusion Models

in28minutes

Getting Started with AI Agents

Why Do We Need AI Agents?

Models are great at understanding, reasoning, and generating outputs

- **Real-world tasks** often require more than generating a response:
  - **Decide** what needs to happen next
  - **Interact** with external systems
  - **Take Actions** based on results
  - **Complete Tasks** involving multiple steps

Question: How do we go one level above models and focus on achieving a specific goal?

- **Answer**: AI Agents

What Are AI Agents?

AI Agent: A system that uses AI models to pursue a goal

- **Observe**: Understand user input and context
- **Reason**: Determine what needs to be done
- **Use Tools**: Access data, invoke API and interact with other applications

Agentic AI: Moves beyond one-off prompts

- Uses models to pursue goals (Can adapt across multiple steps)

Key Idea: Move beyond generating responses to getting things done

AI Agents - An Example

User Request: "Summarize report and send to team"

A model can:

- **Generate**: Summarize the report

An agent can:

- **Understand**: Identify the user's goal
- **Generate**: Create the summary using a model
- **Find Information**: Identify the team members
- **Use Tools**: Connect to email
  - **Take Action**: Send the summary
- **Result**: The task is completed

Use Cases for AI Agents [1/2]

Customer Agents: Assist customers and perform customer-facing tasks

- **Example**: Answer account questions and help resolve issues

Employee Agents: Help employees complete internal tasks

- **Example**: Answer HR questions and submit leave requests

Creative Agents: Assist with content creation workflows

- **Example**: Create campaign copy and visuals

Use Cases for AI Agents [2/2]

Data Agents: Analyze data and generate insights

- **Example**: Analyze sales data and summarize important trends

Coding Agents: Assist with software development tasks

- **Example**: Generate code, review, and fix issues

Security Agents: Assist with detecting and responding to threats

- **Example**: Investigate suspicious activity and initiate remediation

Agent Pattern: Conversational Agents

Goal: Interact with users through natural language

Flow:

- **1: Receive Request**: User requests an action
- **2: Understand**: Agent understands the request
- **3: Use Tools**: Fetch information or perform actions
- **4: Respond**: Return the result to the user

Examples:

- **Healthcare**: "Book an appointment with Dr. XYZ tomorrow"
- **Retail**: "Where is my order?"

Agent Pattern: Workflow Agents

Goal: Execute multi-step processes to achieve a business goal

- **1: Trigger**: User or system initiates a task
- **2: Determine Steps**: Determine what to do
- **3: Execute**: Use tools and external systems
- **4: Adapt**: Determine subsequent steps
- **5: Complete**: Deliver the final result

Examples:

- **HR On-boarding Agent**: Create account → assign equipment → schedule orientation
- **IT Operations Agent**: Diagnose issue → restart service → create incident → notify team

Getting Started with AI Agent Architecture [1/2]

Persona: Defines the agent's role, behavior, and communication style

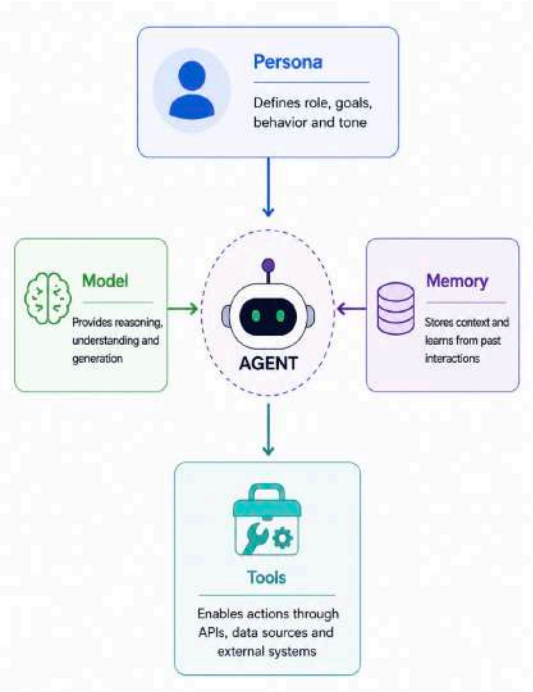
- **Defined Using**: Agent instructions or system prompts

Model (LLM): Provides the core intelligence for the agent

- **Understand**: Comprehend user requests and instructions
- **Reason**: Determine what needs to be done
- **Generate**: Create responses

Memory: Provides context across interactions

- **Short-Term**: Current conversation and task context
- **Long-Term**: Historical information and past interactions



Getting Started with AI Agent Architecture [2/2]

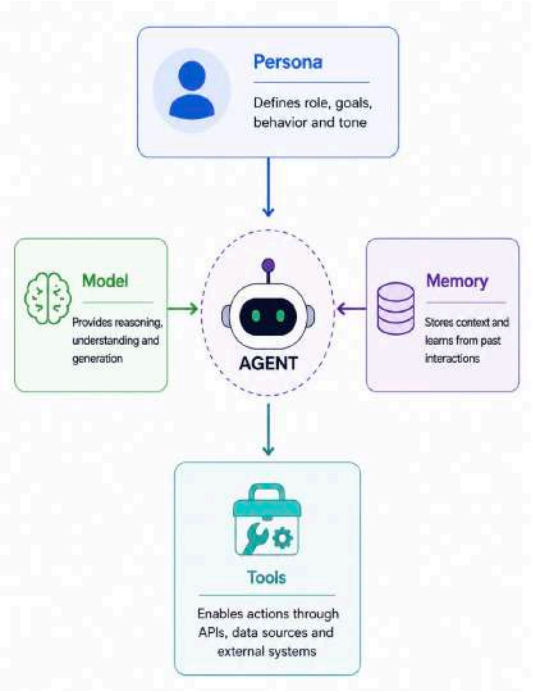
Tools: Extend what an agent can do

Knowledge Tools: Access external information

- **Examples**: Search engines, documents, databases, etc.

Action Tools: Perform tasks in external systems

- **APIs**: Connect to services such as translation or travel booking
- **Functions**: Execute application logic such as `calculate_xyz()`
- **Integrations**: Connect to calendars, email, payment systems, etc



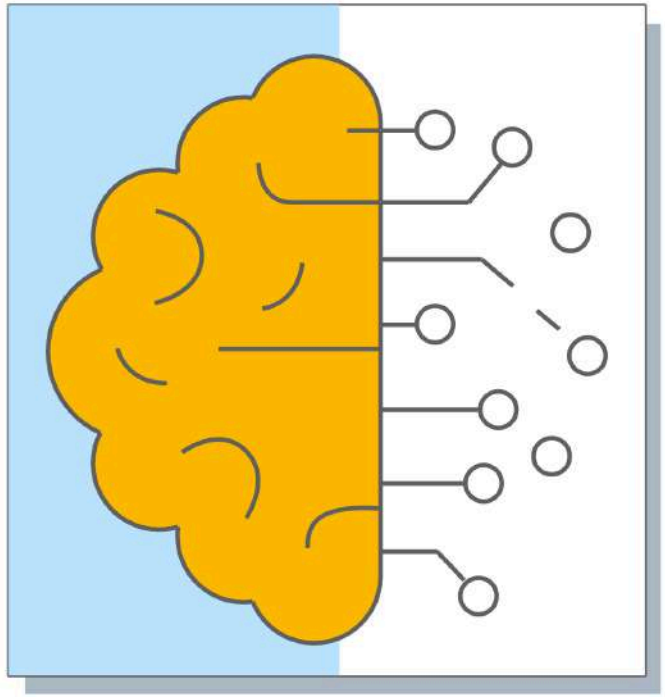
in28minutes

Getting Started with Microsoft Foundry

What is Microsoft Foundry? [1/2]

Microsoft Foundry: Unified Azure platform for building and operating AI apps and agents

- **Models**: Discover, evaluate, customize, and deploy AI models
- **Agents**: Build and deploy agents
- **Tools**: Give the agent capabilities (actions)
- **Knowledge**: Give the agent access to data
- **Enterprise Readiness**: Security, monitoring, governance etc.



Microsoft Foundry: Models + Agents + Tools + Knowledge + Enterprise Readiness

What is Microsoft Foundry? [2/2]

Models: Extensive catalog of **AI models**

- **Examples**: OpenAI GPT, Anthropic Claude, Meta Llama, Mistral, DeepSeek, xAI Grok, etc.

Agents: Build and run AI agents

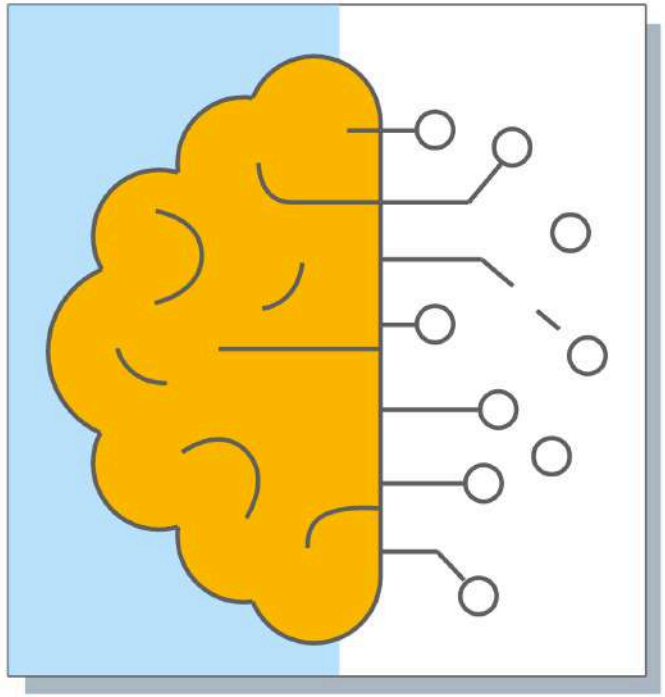
- Use **prompt-based** or **code-first** approaches

Tools: Give agent capabilities

- **Question**: What can the agent **do**?
- **Examples**: Call API, automate actions

Knowledge: Give the agent access to data

- **Question**: What can the agent **know**?
- **Examples**: Azure databases, SharePoint, web content



How is Microsoft Foundry Organized? [DEMO]

Foundry Resource: Top-level Azure resource for Microsoft Foundry

- Manage shared settings such as security, networking, and model deployments
- A Foundry resource can contain **multiple projects**

Foundry Project: Development workspace within a Foundry resource

- Organizes resources and assets for AI applications and agents
- Provides the working context for developers and applications

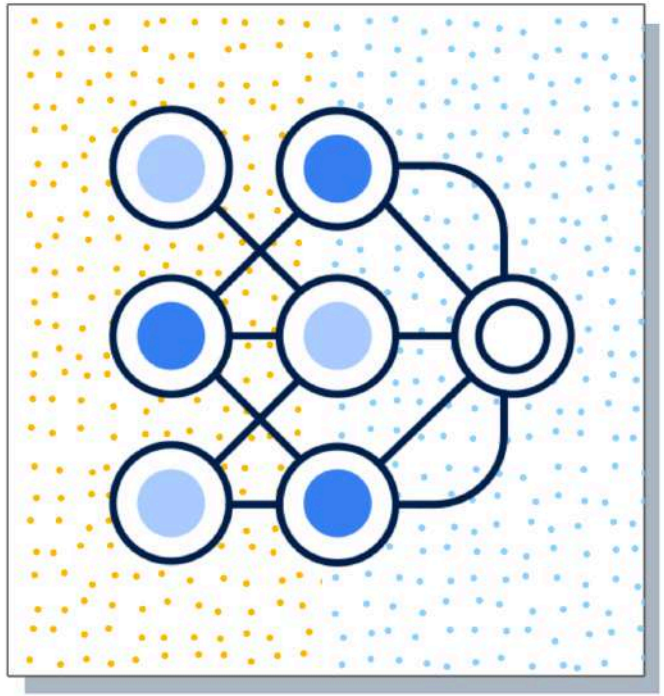
Microsoft Foundry - Exploring Models [DEMO]

Model Catalog: Explore hundreds of models from Microsoft and partners

Model Details: Explore capabilities, use cases, pricing, technical specifications, and deployment options

Compare Models: Compare models across quality, safety, cost, and performance

Model Leaderboard: Identify top-performing models using standard benchmarks



Microsoft Foundry - Deploying a Model [DEMO]

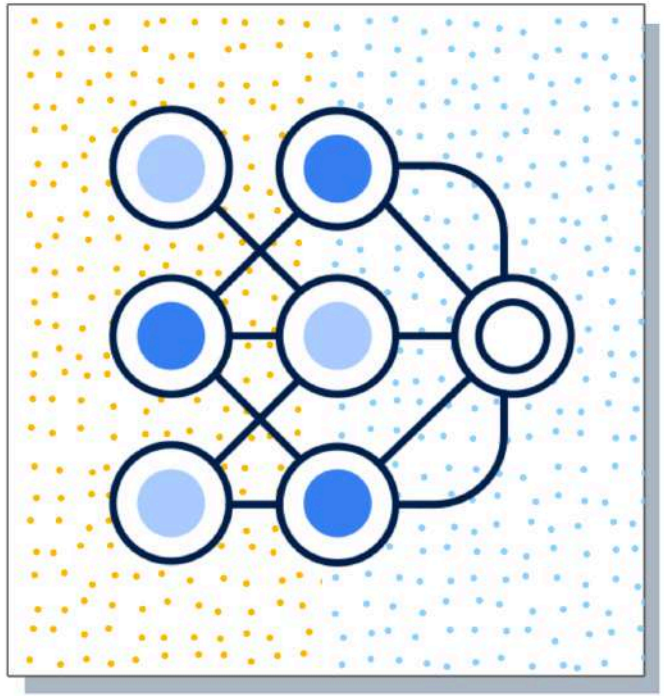
Model: Choose the model to deploy

Deployment Mode: Choose between

- **Default Settings**: Use the recommended defaults OR
- **Custom Settings**: Set your own quota, guardrails etc.

Deployment Options:

- **Standard Deployment**: Foundry manages the underlying infrastructure (For Foundry Models sold by Azure and select partner/community models)
- **Managed Compute**: Run open-source models using dedicated GPU compute (Examples: Hugging Face models, NVIDIA NIMs)



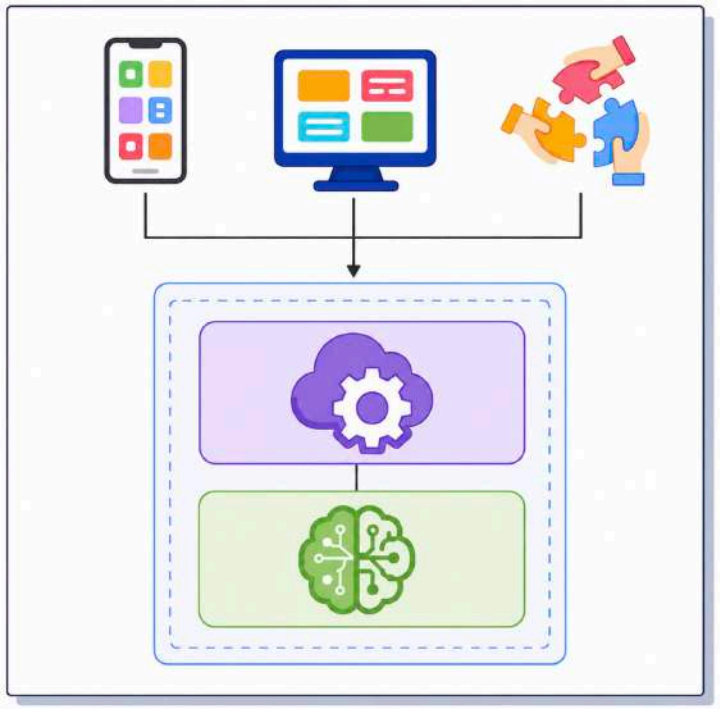
Invoking Generative AI API From Code [DEMO]

Your deployed model is available through a **Foundry API endpoint**

How do you invoke the API from code?

SDKs: Create requests and process responses using simple classes and methods

- Avoid manually creating requests, authentication headers, JSON payloads, and parsing responses
- **OpenAI SDK**: Invoke models that support the OpenAI-compatible API
- **Foundry Tools SDKs**: Work with purpose-built services such as Language and Speech

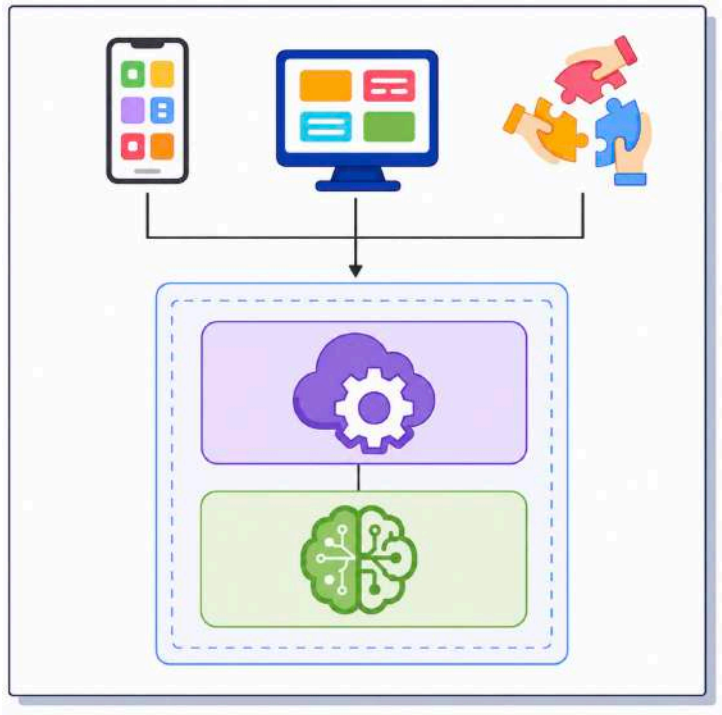


Code Examples: Making API calls using SDKs

High Level Understanding: You need a high level understanding of using the SDKs

Calling API: Typically involves 3 steps

- **Step 1**: Authenticate and get a Client
  - `client = OpenAI(base_url="<your-endpoint>", api_key="<your-api-key>")`
- **Step 2**: Call the API
  - Use the SDK to make a call
  - `response = client.responses.create ( model="<your-deployment-name>", input="<your-prompt>")`
- **Step 3**: Process Response
  - `print(response.output_text)`



OpenAI SDK Example - Sending a prompt through API

`OpenAI()`: Creates the client used to invoke the deployed model

- `<your-endpoint>`: Points to the Foundry model API endpoint
- `<your-api-key>`: Key

Responses API: Modern, unified API for generative AI interactions

- Supports text and multimodal inputs
- Supports conversational workflows

```
# pip install openai [OpenAI SDK]

client = OpenAI(
    base_url="<your-endpoint>",
    api_key="<your-api-key>")

response = client.responses.create(
    model="<your-deployment-name>",
    input="What can you learn with in28minutes?"
)

print(response.output_text)
```

What is Natural Language Processing (NLP)?

Natural Language Processing (NLP): Enables computers to understand, interpret, analyze, and generate human language

- **Customer Reviews**: Identify sentiment and frequently mentioned topics
- **Emails**: Classify messages
- **Documents**: Identify entities (people, locations, organizations, dates, etc.), key phrases, and sensitive information
- **Support Tickets**: Categorize and identify concerns

Natural Language Processing (NLP) - Key Capabilities

Capability	Description	Example
Language Detection	Identify the language used in text	Detect English, Spanish, or French
Text Classification	Categorize text into predefined groups	Complaint, Question, Feedback
Sentiment Analysis	Determine sentiment expressed in text	Positive, Negative, Neutral
Key Phrase Extraction	Identify important phrases and concepts	Extract product names and topics
Named Entity Recognition	Identify people, locations, organizations, dates, etc	Find company and location names
PII Detection / Redaction	Identify or mask sensitive information	Emails, phone numbers, account numbers
Summarization	Create concise summaries of text	Summarize reports or reviews

Natural Language Processing Workloads - Scenarios

Scenario	Solution
Analyze thousands of customer reviews and determine whether each is positive, negative, or neutral	Sentiment Analysis
Identify the most important topics and phrases from customer reviews	NLP - Key Phrase Extraction
Identify people, organizations, locations, and dates mentioned in news articles	Entity Recognition
Create a short overview of a long customer report	Summarization

Traditional Text Processing Techniques

Text Normalization: Standardize text

- "Hello, WORLD!" → "hello world"

Stop Word Removal: Remove common words

- "The cat sat on a mat" → "cat sat mat"

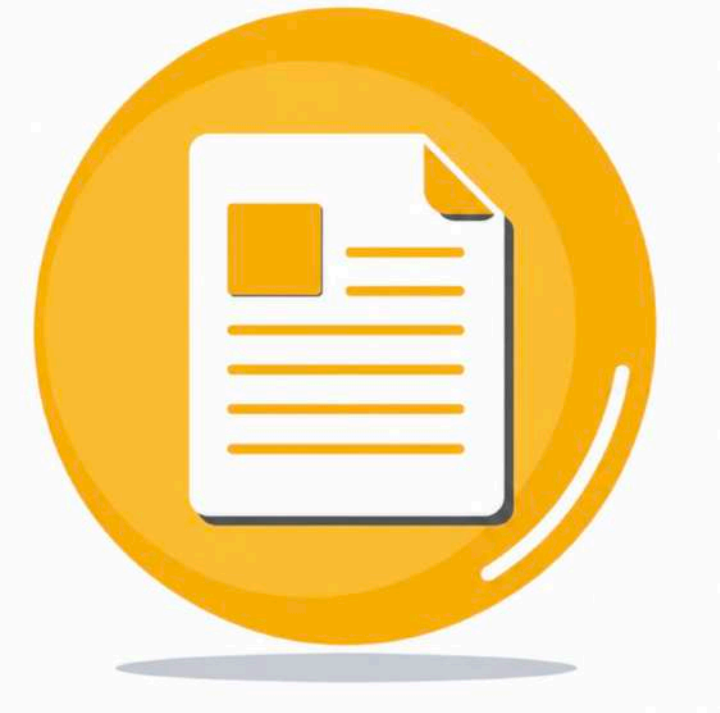
Stemming: Reduce to a base form

- "power", "powered", "powerful" → "power"

n-grams: Analyze word sequences together

- **Bigram**: "machine learning"
- **Trigram**: "natural language processing"

Frequency Analysis: Identify frequently occurring words or phrases



Traditional Text Processing Techniques - An Example

Original Text: "The customers were happily using their new phones!"

- **Text Normalization**: Standardize the text
  - "the customers were happily using their new phones"
- **Stop Word Removal**: Remove common words
  - "customers happily using new phones"
- **Stemming**: Reduce to a base form
  - "customer happy use new phone"
- **n-grams**: Analyze word sequences together
  - **Bigram**: "new phone"
- **Frequency Analysis**: Identify frequently occurring words or phrases

Traditional Text Processing Techniques - Scenarios

Scenario	Solution
Convert "Hello, WORLD!" into "hello world" before analysis	Text Normalization
Remove words such as "the", "a", and "is" when they add little value to the analysis	Stop Word Removal
Treat "connect", "connected", and "connecting" as related forms	Stemming
Analyze phrases such as "credit card" or "machine learning" instead of individual words	n-grams
Find the most frequently occurring words or phrases across thousands of reviews	Frequency Analysis
Identify product names and determine which are mentioned most frequently	Entity Recognition + Frequency Analysis

Text Analysis in Microsoft Foundry [DEMO]

General-Purpose AI Models: Perform text analysis using natural language prompts [GPT]

- **Examples**: Key phrase extraction, entity recognition, sentiment analysis, summarization, and classification

Purpose-Built Language Tools: Provide specialized capabilities

- **Azure Language in Foundry Tools** - Language detection, PII detection, sentiment analysis, entity recognition, etc.

OpenAI Example - Text Analysis

OpenAI SDK: Use Generative AI models for flexible text analysis

Prompt Customization: Define the task using natural-language instructions

`OpenAI()`: Creates the client used to invoke the deployed model

Responses API: Modern unified API for Generative AI interactions

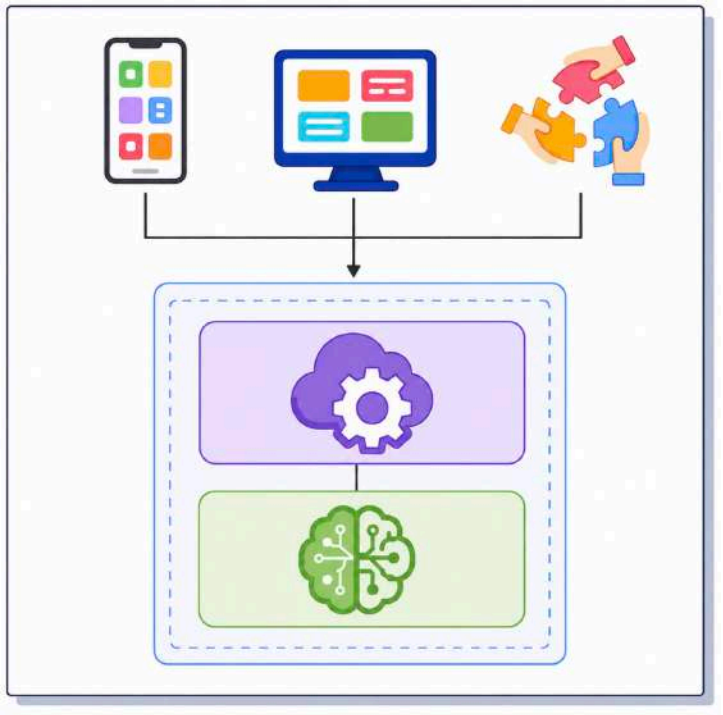
```
# pip install openai [OpenAI SDK]

client = OpenAI(
    base_url="<your-endpoint>",
    api_key="<your-api-key>")

response = client.responses.create(
    model="<your-deployment-name>",
    input="Identify sentiment as Positive or Negative:
<<REVIEW CONTENT>>"
)

# Change the prompt for different text analysis tasks:
# Key Phrases: "Extract the key phrases from: <<TEXT>>"
# Entities: "Identify people and locations in: <<TEXT>>"
# Summarization: "Summarize in 3 bullet points: <<TEXT>>"

print(response.output_text)
```

Getting Started with Foundry SDK

Foundry SDK: Integrate models, agents, and other Foundry capabilities into your agents and applications

Foundry Tools SDKs: SDKs for specialized, prebuilt AI services

- Examples: Language, Vision, Speech, Document Intelligence
- Use when working directly with a specific AI capability



Azure Language SDK Example - Detect Language

Azure Language SDK: Purpose-built SDK for text analysis

`TextAnalyticsClient`: Client used to invoke Language capabilities

- **Endpoint**: URL of the Azure Language resource
- **API Key**: Used to authenticate the request
- `detect_language()`: Detect the language of the input text

Key Idea: Use a specialized AI service for a predefined task

```
# We are using Azure Language SDK
# Client library for Azure Language in Foundry Tools
# pip install azure-ai-textanalytics

text_analytics_client = TextAnalyticsClient(
            endpoint="<your-endpoint>",
            credential=AzureKeyCredential("<your-api-key>"))

text = "Hello! This is Ranga from in28minutes."

response = text_analytics_client.detect_language(
    documents=[text]
)[0]

print("Language:", response.primary_language.name)
```

TextAnalyticsClient - Other Key Methods

Operation	Method	Example
Sentiment Analysis	*analyze_sentiment()*	Determine whether a product review is positive, negative, or neutral
Named Entity Recognition	*recognize_entities()*	Identify people, organizations, locations, dates, etc
PII Recognition	*recognize_pii_entities()*	Detect sensitive information such as phone numbers, emails, and IDs
Key Phrase Extraction	*extract_key_phrases()*	Extract important phrases and topics from customer feedback

in28minutes

Getting Started with Azure Speech

Azure Speech Services - A Quick Overview

Service	Description
Speech to Text (speech recognition)	Convert spoken audio into text for transcription, captions, voicemail processing, and AI-agent input (Live from a microphone or use an audio file)
Text to Speech (speech synthesis)	Convert text into human voice
Text to Speech Avatar	Convert text into video featuring a photo-realistic avatar
Speech Translation	Translate spoken audio across languages into translated text or speech
Voice Live	Enable low-latency, speech-to-speech conversations for voice agents

Exploring Azure Speech Use Cases

Use Case	Description
Captioning	Generate captions from audio for videos, meetings, and multilingual content
Audio Content Creation	Convert text such as e-books into natural-sounding speech
Call Center	Transcribe calls and analyze interactions for insights such as sentiment
Language Learning	Assess pronunciation, transcribe conversations, and read learning content aloud
Real Time Voice Interactions	Build real-time, natural voice interactions between users and AI agents using Voice Live
Speech Translation	Translate speech into other languages in real time
Video Creation	Create photo-realistic talking-avatar videos for real-time or batch applications

Azure Speech - Speech to Text (Speech Recognition)

Speech to Text: Convert spoken audio into written text

Two common approaches:

- **Real-Time Transcription**: Transcribe live audio streams with results available as the audio is received
  - **Example**: Generate live transcription for a presentation or meeting
- **Batch Transcription**: Transcribe large volumes of prerecorded audio asynchronously
  - **Example**: Process call recordings stored in Azure Storage

Key Idea: Use real-time transcription for **live audio** and batch transcription for **prerecorded audio**

Azure Speech - Speech to Text - Code Example [1/3]

Speech SDK: For Speech Apps

SpeechConfig: Configure connection to Azure Speech

- **Endpoint**: Identifies the endpoint
- **Key**: Authenticates the request

Audio Configuration: Defines where the audio comes from

- In this example: **default microphone**
- Also possible: From a file

Speech Recognizer: Combines speech config and audio input

```
# pip install azure-cognitiveservices-speech

# Configure Azure Speech
speech_config = speechsdk.SpeechConfig(
    subscription="<your-api-key>",
    endpoint="<your-endpoint>"
)

# Configure microphone as audio input
audio_config = speechsdk.audio.AudioConfig(
    use_default_microphone=True
    # filename="path_to_file" # FILE AS INPUT
)

# Speech recognizer: Speech to Text Capability
speech_recognizer = speechsdk.SpeechRecognizer(
    speech_config=speech_config,
    audio_config=audio_config
)
```

Azure Speech - Speech to Text - Code Example [2/3]

Define: Create event handlers

- **Recognizing**: Handles intermediate transcription
- **Recognized**: Handles final transcription for the current utterance

Connect: Connect event handlers to SpeechRecognizer

```
# Define Handlers
def recognizing_handler(evt):
        print(f"Recognizing: {evt.result.text}")

def recognized_handler(evt):
        print(f"Recognized: {evt.result.text}")

# Connect Handlers
speech_recognizer.recognizing.connect(
                        recognizing_handler)
speech_recognizer.recognized.connect(
                        recognized_handler)

#Recognizing: Hello
#Recognizing: Hello everyone
#Recognizing: Hello everyone welcome
#Recognizing: Hello everyone welcome to Foundry
#Recognized: Hello everyone, welcome to Foundry.
```

Azure Speech - Speech to Text - Code Example [3/3]

Question: Does listening start after handlers are connected?

- **Answer**: No

Why?: You need to kickstart recognition process

- `start_continuous_recognition()`: Starts continuous speech recognition for multiple utterances
- `stop_continuous_recognition()`: Stops continuous speech recognition

```
speech_recognizer.recognizing.connect(
                    recognizing_handler)
speech_recognizer.recognized.connect(
                    recognized_handler)

# Start continuous recognition
speech_recognizer.start_continuous_recognition()
print("Say something...")
input("Press Enter to stop...")
speech_recognizer.stop_continuous_recognition()


#Recognizing: Hello
#Recognizing: Hello everyone
#Recognizing: Hello everyone welcome
#Recognizing: Hello everyone welcome to Foundry
#Recognized: Hello everyone, welcome to Foundry.
```

SpeechRecognizer - Common Methods

Method	Description
recognize_once()	Recognizes a single utterance and returns when the utterance ends
start_continuous_recognition()	Starts continuous speech recognition for multiple utterances
stop_continuous_recognition()	Stops continuous speech recognition
start_keyword_recognition()	Starts listening for a configured keyword and begins recognition when the keyword is detected
stop_keyword_recognition()	Stops keyword-triggered speech recognition
recognize_once_async() **and other _async methods**	Perform operations asynchronously

Azure Speech - Text to Speech - Code Example [1/2]

Speech SDK: For building speech applications

SpeechConfig: Connect your app to Azure Speech

- **Endpoint**: Identifies the Speech service endpoint
- **Key**: Authenticates the request

Audio Output: Defines where synthesized audio goes

- In this example: **default speaker**
- Also possible: **audio file**

```
# pip install azure-cognitiveservices-speech

# Configure Azure Speech
speech_config = speechsdk.SpeechConfig(
    subscription="<your-api-key>",
    endpoint="<your-endpoint>"
)

# use the default speaker as audio output.
audio_config = speechsdk.audio.AudioOutputConfig(
    use_default_speaker=True)

#
speechsdk.audio.AudioOutputConfig(filename="output.wav")
```

Azure Speech - Text to Speech - Code Example [2/2]

Speech Synthesizer: Combines speech config and audio input

speak_text(): Synthesize plain text into speech

```
# List of voices - https://aka.ms/speech/voices/neural
speech_config.speech_synthesis_voice_name
    = "en-US-Ava:DragonHDLatestNeural"


speech_synthesizer = speechsdk.SpeechSynthesizer(
                        speech_config=speech_config,
                        audio_config=audio_config)

text = "Hello, welcome to Azure AI Foundry!"

result = speech_synthesizer.speak_text(text)
```

Azure Speech - Voice Live [1/2]

Building a **real-time voice agent** typically requires:

- **Speech to Text**: Understand what the user says
- **AI Model / Agent**: Reason and determine the response
- **Text to Speech**: Respond with spoken audio

What if these capabilities are integrated into **one experience**?

Voice Live: Build low-latency, real-time voice agents with speech, AI, and agent capabilities integrated into one experience

Azure Speech - Voice Live [2/2]

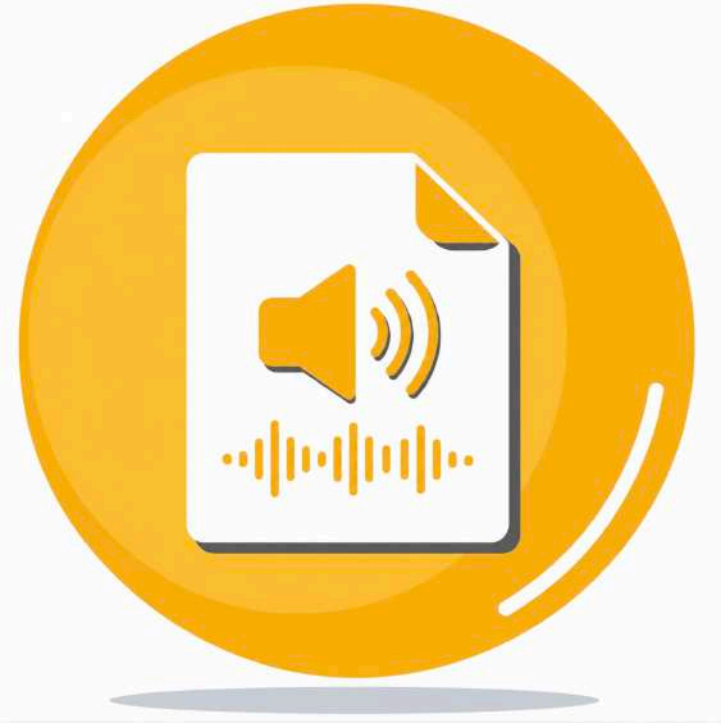
Natural Conversation: Enable real-time speech-to-speech interactions

- **Low Latency**: Support real-time conversations
- **Actions**: Enable agents to use tools and perform actions
- **Avatars**: Add visual avatar experiences

Foundry Integration:

- Explore Voice Live using the **Foundry playground**
- Enable **Voice mode** for Foundry agents

Key Idea: Voice Live simplifies **Speech → AI → Speech** experiences



in28minutes

Getting Started with Images and Videos

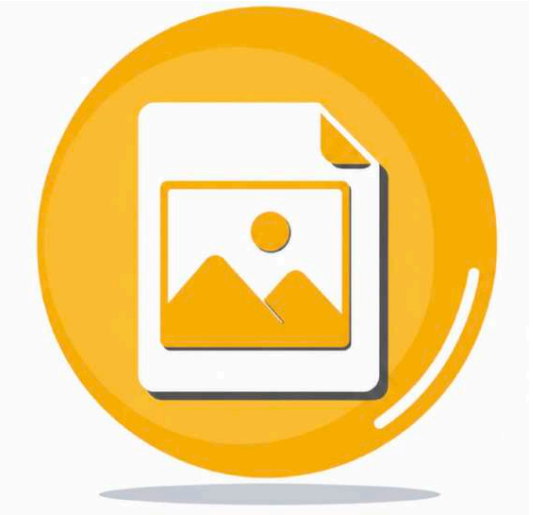
Getting Intelligence From Images - Terminology

Image Analysis: Generate tags and descriptions

Image Classification: Assign a category to an image

Object Detection: Identify objects in image

- For each object: bounding box and a confidence score
- Goes deeper than image classification
- **Semantic Segmentation**: Go even deeper
  - Identify pixels that belong to a particular object



Face Detection: Detect human faces

Optical Character Recognition (OCR): Detect and extract text from images

- Examples: License plates, receipts, invoices

Getting Intelligence From Images - Terminology - Scenarios

Scenario	Terminology
Generate captions for a photo	Image Analysis
Sort images into categories like ‘food’, ‘animals’, or ‘landscapes’	Image Classification
Identify cars, bikes, and pedestrians in a street image with bounding boxes	Object Detection
Identify every pixel belonging to the tree in a forest photo	Semantic Segmentation
Recognize and extract text from a scanned receipt	Optical Character Recognition (OCR)
Detect and locate faces in a group photo	Face Detection

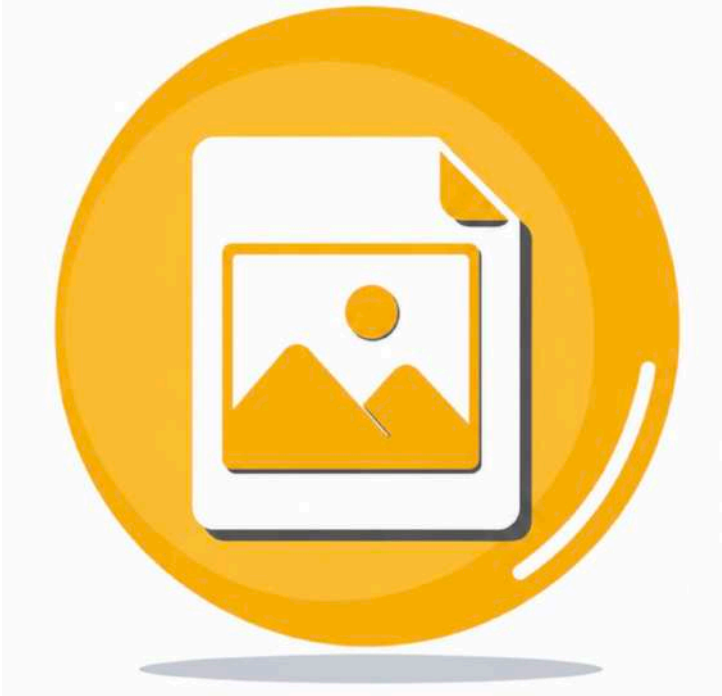
Getting Started with Vision Models [DEMO]

Dedicated Vision Models: Focused on visual content

- **Examples**: **GPT Image** models, **Sora** video models

Multimodal Models: Multiple types of inputs/outputs

- **Modalities**: Subset of Text, images, audio, video
- **Examples**: Vision-capable GPT models



Example 1 - Image Analysis

Pre-requisite: Deploy a vision-capable GPT model

Multimodal Input: Send text instructions and an image in the same request

- `input_text`: Provides instructions to the model
- `input_image`: Provides the image to analyze
- **Image Input**: Can use an image URL or Base64-encoded image

```
# client = OpenAI(..)
image_url = "<<image-url>>"

response = client.responses.create(
    model="<<your-deployment-name>>",
    input=[
        {
            "type": "input_text",
            "text": "Describe the image in 20 words"
        },
        {
            "type": "input_image",
            "image_url": "<<image-url>>"
        }])

print(response.output_text)

# Using Local Image - Base64
# base64_image = encode_image("image_path.jpg")
# "image_url":f"{base64_image}"
```

Example 2 - Image Generation

Image Generation: Create images from natural language instructions

Pre-requisite: Deploy a GPT Image model for image generation

```
prompt = "Create an image of a futuristic city"

response = client.responses.create(
    model="<<your-deployment-name>>",
    input=prompt,
    tools=[{"type": "image_generation"}]
)

# Get generated image
image_base64 = response.output[0].result

# Save image
with open("generated_image.png", "wb") as file:
    file.write(base64.b64decode(image_base64))

print("Saved: generated_image.png")
```

Example 3 - Video generation with Sora

Sora 2: Generate videos

- **Text → Video**: Generate a new video from a prompt
- **Image → Video**: Use an image as the starting point
- **Video → Video**: Remix an existing generated video

Asynchronous: Video generation takes time

- **1: Create** a video generation job
- **2: Check** the job status
- **3: Download** the completed video

```
# 1. Create video
POST <endpoint>/openai/v1/videos

# 2. Check status
GET <endpoint>/openai/v1/videos/<video-id>

# 3. Download completed video
GET <endpoint>/openai/v1/videos/<video-id>/content
```

in28minutes

Getting Started with Azure Content Understanding

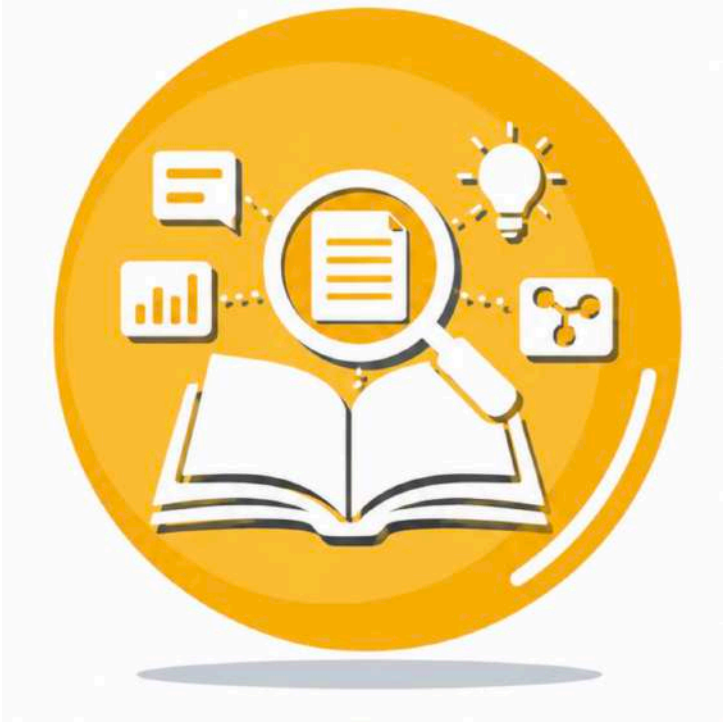
Getting Started with Azure Content Understanding

Azure Content Understanding: Extract structured information from different types of unstructured content

Supports multiple **modalities**:

- **Documents & Images**: Extract information from PDFs, forms, invoices, receipts, contracts, and images
- **Audio**: Extract information from calls, recordings, and other audio content
- **Video**: Extract information from meetings, presentations, and other video content

Key Idea: Unstructured Content → Structured Information



Azure Content Understanding - How does it work?

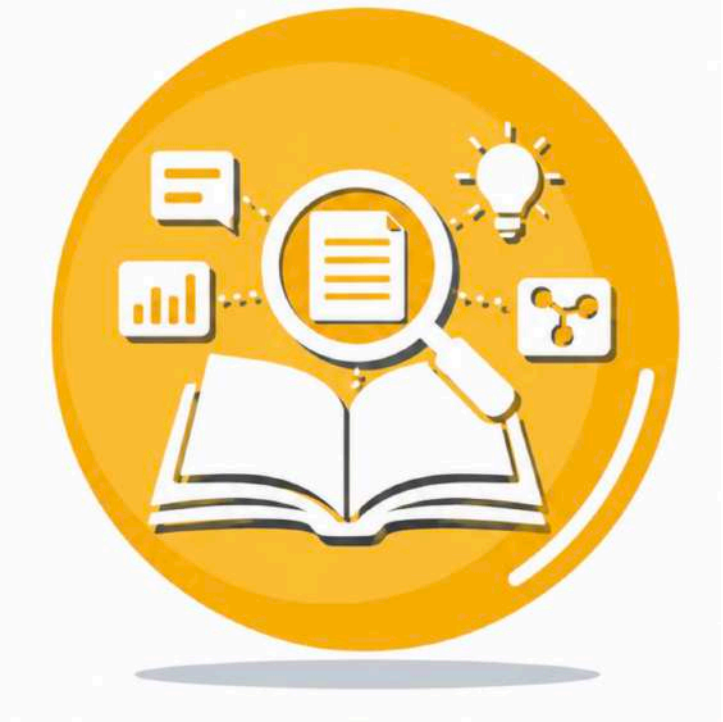
Input: Documents, images, audio, video files

Analyzer: Defines the processing to be done

- **Content Extraction**: Extract information
  - **Documents / Images**: Extract text using OCR and recognize layout elements such as paragraphs, sections, and tables
  - **Audio / Video**: Transcribe speech, summarize visuals
- **Field Extraction**: Extract fields [pre-defined schema]
  - **Output**: Key-value pairs [`InvoiceNumber`, `Date`, and `Total`]
- **Segmentation**: Divide content into sections
  - **Examples**: Divide videos into scenes

Output: JSON [with markdown summary]

- Useful for application integration and search



Azure Content Understanding - Schema Based Field Extraction

Schema: Defines what information to extract and how it should be structured

Supports **fields, nested fields, and collections**

- **Example**: Invoice Number, items [collection of description, quantity, unit price, and total]

Azure Content Understanding maps information from the content to the **schema fields**

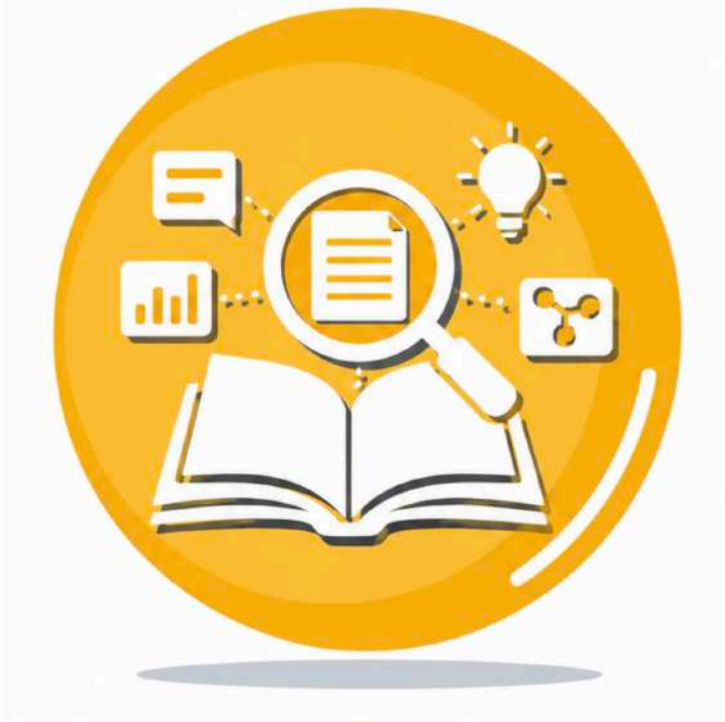
Azure Content Understanding - Analyzers [1/3]

Base Analyzers: General processing for different content types

- `prebuilt-audio`: Process audio
- `prebuilt-document`: Process documents
- `prebuilt-image`: Process images
- `prebuilt-video`: Process videos

Content extraction analyzers: Focus on OCR and Layout

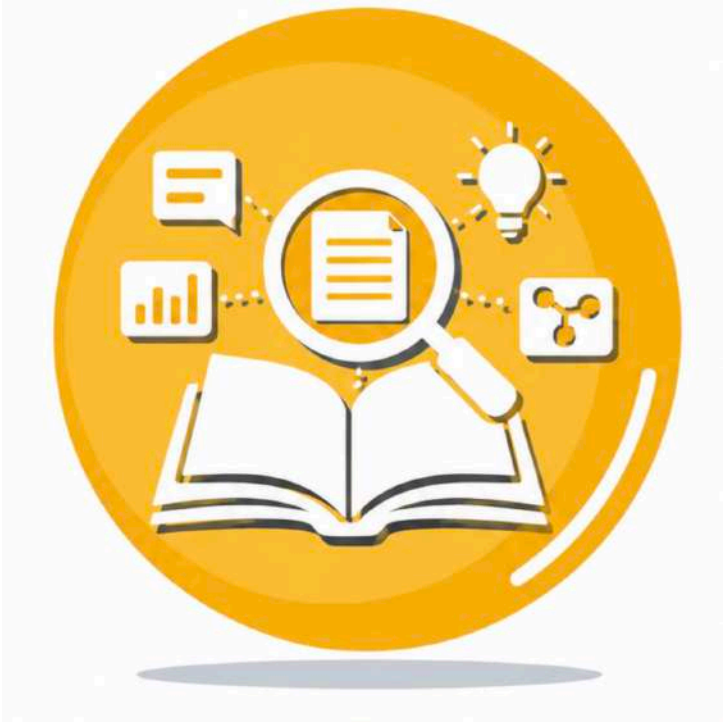
- `prebuilt-read`: OCR/text extraction without layout analysis
- `prebuilt-layout`: Text plus layout/document structure, including figures, tables, sections, formatting etc.



Azure Content Understanding - Analyzers [2/3]

RAG Analyzers: Prepare multimodal content for search and retrieval

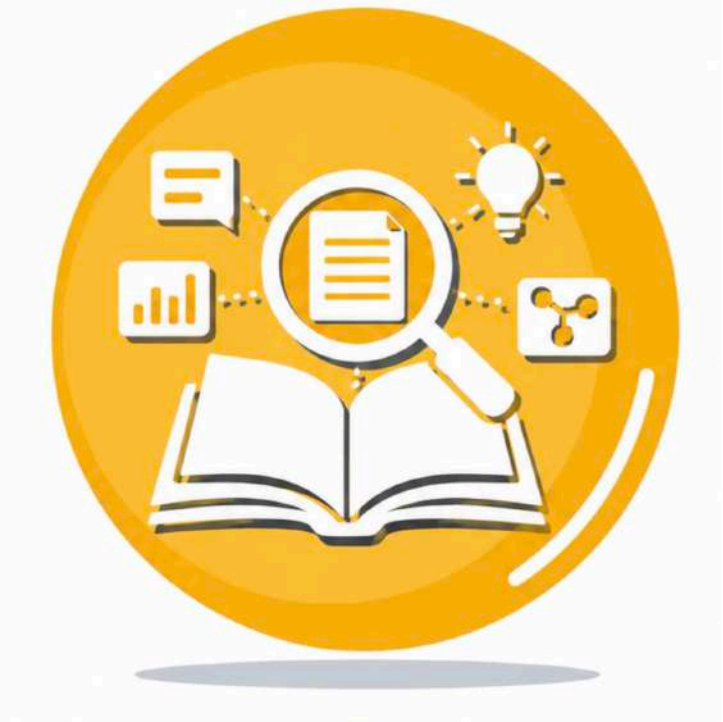
- `prebuilt-documentSearch`: Extract document content, layout and descriptions of visual elements
- `prebuilt-imageSearch`: Generate descriptions and insights from images
- `prebuilt-audioSearch`: Transcribe conversations from audio
- `prebuilt-videoSearch`: Extract transcripts and descriptions from video segments
- `prebuilt-callCenter`: Extract insights such as topics and sentiment from call recordings



Azure Content Understanding - Analyzers [3/3]

Domain-Specific Analyzers: Extract predefined fields from common business documents

- `prebuilt-invoice`: Invoices, purchase and sales orders
- `prebuilt-receipt`: Retail and restaurant receipts
- `prebuilt-utilityBill`: Electricity, water, gas, internet, and phone bills
- `prebuilt-tax.us`: Classify and extract information from US tax forms
- `prebuilt-idDocument`: Passports, driver licenses, ID cards, etc.
- `prebuilt-creditCard`: Credit card statements



Azure Content Understanding - Code Example

`ContentUnderstandingClient`: Connects to Azure Content Understanding

Analyzer: Defines how the content should be analyzed

Input: Provide the document, image, audio, or video to analyze

`begin_analyze()`: Starts the analysis operation

`result()`: Waits for processing to complete and returns the result

```
# pip install azure-ai-contentunderstanding
client = ContentUnderstandingClient(
    endpoint="<<endpoint>>",
    credential=AzureKeyCredential("<<key>>"))

analyzer_id = "prebuilt-invoice" # "prebuilt-audioSearch", ..

inputs = [{"url": "URL_TO_PDF_OR_IMAGE"}] #Your content

poller = client.begin_analyze(
    analyzer_id=analyzer_id, inputs=inputs)

#wait for analysis to complete
result = poller.result()

# read structured fields + markdown
# The result typically includes
# extracted "fields" and "markdown" per input content item.
for content in result.contents:
        print(content.markdown)
        print(content.fields)
```

Azure Content Understanding - Simplified Example Response


```
{
  "WARNING": "NOT COMPLETE",
  "id": "some-id",
  "status": "Succeeded",
  "result": {
    "analyzerId": "my-analyzer",
    "contents": [{
        "markdown": "# Example Document\n\n...",
        "fields": { /* extracted field values */ },
        "kind": "document",
        "startPageNumber": 1, "endPageNumber": 2,
        "pages": [ /* page-level elements */ ],"paragraphs": [ /* paragraph elements */ ],"sections": [ /* section elements */ ],
        "tables": [ /* table elements */ ],"figures": [ /* figure elements */ ],"hyperlinks": [ /* hyperlink elements */ ],
        "metadata": { /* document metadata */ }
        /* kind: audioVisual - KeyFrameTimesMs,transcriptPhrases with startTimeMs, endTimeMs and text */
        /*prebuilt-callCenter - Topics: Companies,People,Sentiment,Categories*/
      }
    ]
  }
}
```

in28minutes

Getting Started with AI Agents - Azure

Exploring Agents in Microsoft Foundry

Prompt Agents: Fastest way to create an agent

- Configure persona, model, and tools
- No application code or infrastructure to manage
- **Foundry**: Runs and manages the agent for you

Hosted Agents: Maximum control

- Bring your own code (Ex: built using Microsoft Agent Framework)
- **Foundry**: Runs it as a managed application
- Provides endpoint, scaling, and other enterprise-ready features

Demo - Prompt Agents

Instructions:

You are a travel planning agent.

Help users plan practical trips based on their interests, budget, and available time.

When creating an itinerary:
- Organize recommendations by day
- Group nearby attractions together
- Include approximate travel time between major stops
- Prefer budget-friendly options when requested
- Keep recommendations concise

Conversation:

I'm visiting Paris for 2 days. Plan my trip.
I'm traveling with two children. Adjust the plan.
How much should I budget for attractions?

Example - Invoke an Agent From Your Application

`AIProjectClient`: Connects to your Foundry project

- `endpoint`: Foundry project endpoint
- `DefaultAzureCredential()`: Authenticates with Azure

`get_openai_client()`: Creates an OpenAI client

`agent_reference`: Identifies the agent to invoke

Responses API: Sends a prompt and gets the response

```
# pip install azure-ai-projects azure-identity

project_client = AIProjectClient(
    endpoint="<<project-endpoint>>",
    credential=DefaultAzureCredential()
)

openai_client = project_client.get_openai_client(
    agent_name="<<your-agent-name>>"
)

response = openai_client.responses.create(
    input="What should the agent do?"
    # extra_body={ "agent_reference": {
    #   "name": "<<your-agent-name>>",
    #   "type": "agent_reference" } }
)

print(response.output_text)
```

Flexibility of OpenAI Responses API [1/2]

```
# 1. Chat
response = client.responses.create(
    model="<<your-deployment-name>>",
    instructions="You are a concise travel expert",
    input="Give me the most famous landmark in Paris"
)
print(response.output_text)

# 2. Generate an image
response = client.responses.create(
    model="<<your-deployment-name>>",
    previous_response_id=response.id,  # Continue conversation
    input="Create an image of it",
    tools=[{"type": "image_generation"}]
)

# 3. Invoke an agent
response = client.responses.create(
    input="Plan a 2-day trip to visit this landmark",
    extra_body={
        "agent_reference": {"name": "<<travel-agent>>","type": "agent_reference"} # Which agent
    }
)
```

Flexibility of OpenAI Responses API [2/2]

```
# Create a conversation
conversation = openai.conversations.create()

# First message
response = openai.responses.create(
    conversation=conversation.id,
    input="What is the size of France in square miles?"
)
print(response.output_text)

# Continue conversation
response = openai.responses.create(
    conversation=conversation.id,
    input="And what is the capital city?"
)
print(response.output_text)
```

Chat Completions API - An older option

Still Supported:
Common in older applications

New Applications:
Responses API is recommended

```
# Chat Completions API
response = client.chat.completions.create(
    model="<<your-deployment-name>>",
    messages=[
        {
            "role": "system",
            "content": "You are a concise travel expert"
        },
        {
            "role": "user",
            "content": "Give me the most famous landmark in Paris"
        }
    ]
)
#response = client.responses.create(
#    model="<<your-deployment-name>>",
#    instructions="You are a concise travel expert",
#    input="Give me the most famous landmark in Paris"
#)
```

in28minutes

Getting Started with Secure AI

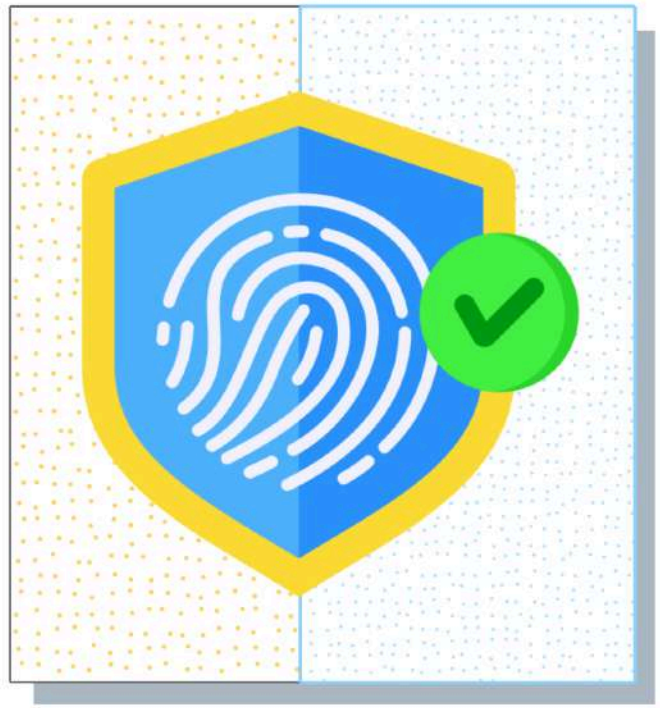
Why Secure AI at Every Stage?

End-to-End Security: AI systems are only as secure as their weakest point

Across the Lifecycle: Security must span data, development, deployment, and operations

Continuous Process: Plan → Secure → Monitor → Repeat

Next: Let's explore how to secure each stage of the AI lifecycle



Secure AI - At Each Stage [1/2]

Secure AI: Security matters throughout AI lifecycle

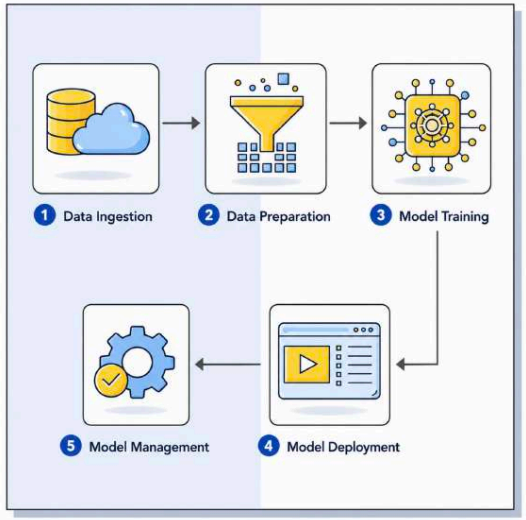
1: Data Ingestion: Secure how data enters the system

2: Data Preparation: Protect and validate data before use

3: Model Training: Secure training data, compute, and model artifacts

4: Model Deployment: Protect endpoints, access, and runtime environments

5: Model Management: Monitor, govern, and update models after deployment



Secure AI - At Each Stage [2/2]

Stage	Key Security Actions
Data Ingestion	Restrict who can upload or modify data. Use trusted sources. Log ingestion activity.
Data Preparation	Mask or anonymize sensitive information. Encrypt data. Validate and clean data before use.
Model Training	Secure the training environment. Restrict access to training data and model configuration. Protect model artifacts from theft or tampering.
Model Deployment	Control who can access the model. Secure APIs with authentication, authorization, and rate limits. Monitor usage.
Model Management	Monitor performance, drift, bias, and security events. Use alerts and update models when needed.

Secure AI – Match the Action to the Stage

Action	Stage
Log every file that gets uploaded to the training folder	Data Ingestion
Prevent unauthorized access to a pricing model's API	Model Deployment
Alert team when prediction accuracy suddenly drops	Model Management
Train models in an isolated environment to avoid external access	Model Training
Replace email addresses with masked IDs before feeding into model	Data Preparation
Detect bias over time and update model configuration	Model Management

in28minutes

Getting Started with Responsible AI - Microsoft

What is Responsible AI?

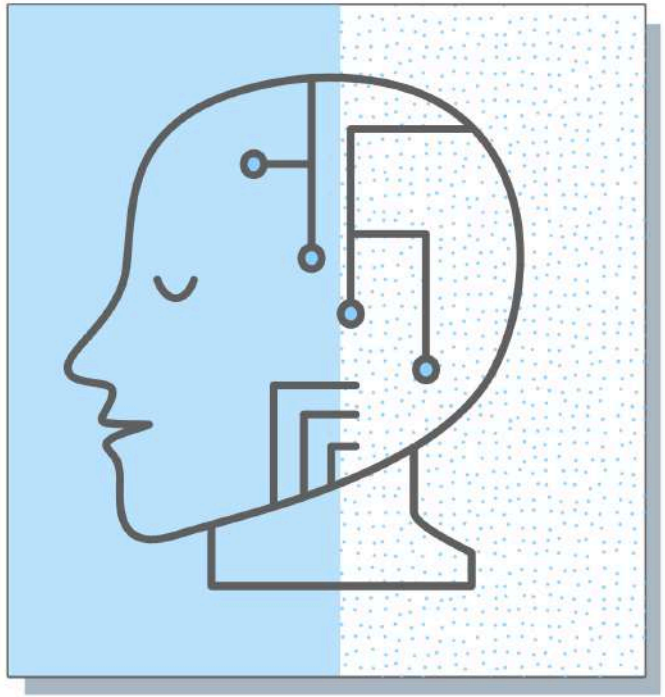
Responsible AI: Practice of designing, developing, and using AI systems in an ethical, secure, and trustworthy manner

- **Goal**: Ensure AI creates value without causing unfair, unsafe, or unintended harm

Focus: NOT limited to the AI model

- **Entire Process**: Data collection, Training, Deployment and Management

Key Questions: Can you trust the AI system and its decisions? Is there someone responsible for AI's decisions?



Responsible AI Principles - Microsoft [1/2]

Principle	What Does It Mean?	Enterprise Example	Remember
Fairness	Fair to all groups of people	A loan approval model should not disadvantage applicants based on gender, race, religion, age, or disability	Data should reflect diversity, Model should evolve with time
Reliability and safety	AI systems should operate consistently, safely, and predictably under normal and unexpected conditions	An autonomous vehicle should respond safely during bad weather, GPS failure, sensor failure, or incorrect input data	Test expected, unexpected (edge scenarios), and failure scenarios
Privacy and security	AI systems should protect personal data, business data, and access to models and services	A healthcare AI model should not expose patient data	Important consideration from day ZERO! Security is important during every stage of ML lifecycle

Responsible AI Principles - Microsoft [2/2]

Principle	What Does It Mean?	Enterprise Example	Remember
Inclusiveness	AI systems should empower people with different abilities	Violation: An application lacks text-to-voice support for visually impaired users	Nobody left out
Transparency	Users should understand when AI is being used, how it influences outcomes, and what its limitations are	A customer should know that an AI system recommended a product and understand the major factors behind the recommendation	Make AI decisions understandable and traceable (Explainability)
Accountability	Organizations and people remain responsible for AI systems	A bank must define who reviews the model, approves its use, handles problems, and responds when the system causes harm	AI is NOT the final decision maker - An enterprise, a team or a person is

Responsible AI - Simple Questions

Principle	Questions to Ask
Fairness	Is the AI treating everyone (all groups) equitably?
Reliability and Safety	Will the AI behave safely when something goes wrong?
Privacy and Security	Is personal and enterprise data protected? Is the system secure?
Inclusiveness	Can people with different abilities use the system effectively?
Transparency	Can users understand how and why the AI produced an outcome?
Accountability	Who is responsible for the AI system and its impact?

Responsible AI Principles - Scenarios [1/2]

Scenario	Solution
Identify violated principle: You find that a ML model does not grant loans to people of certain gender	Fairness
Identify violated principle: More accidents caused by a self driving car in bad weather	Reliability and Safety
Identify related principle: Securing data used to create the model	Privacy and Security
Identify related principle: Making sure that the dataset used does not have any errors (missing values etc)	Reliability and Safety
Identify violated principle: People with disabilities cannot use a specific AI solution	Inclusiveness

Responsible AI Principles - Scenarios [2/2]

Scenario	Solution
Identify related principle: Giving your customers control/choice over the data that is used by your AI system	Privacy and Security
Identify related principle: Ensuring that an AI system works reliably under unexpected situation	Reliability and Safety
Identify violated principle: You do not know how a AI system reached a specific inference	Transparency
Identify related principle: Ensuring that there is sufficient information to debug problems with an AI system	Transparency
Identify related principle: Having a team that can override decision made by an AI system	Accountability

in28minutes

Getting Ready For the Certification Exam

What to Expect in the Exam

Exam Format: 40-60 questions

Question Types:

- **Type 1** : One Answer - 2/3/4 options & 1 right answer
- **Type 2** : Multiple Answers - 5 (or more) options and 2 (or more) right answers

No Penalty: Wrong answers do not reduce your score

Instant Result: Result shown immediately after submitting

- **Email Confirmation**: Sent within a few days

Certification Exam - My Recommendations

Read the Entire Question

- Identify the **key requirements and constraints**
- Read **all answer options** before choosing

Eliminate Wrong Answers

- Remove clearly incorrect options first

Mark and Review

- Mark difficult questions for review
- Revisit them before final submission

Manage Your Time

- Do not spend too much time on one question
- Move on and return later if needed

Get Ready For Your Exam

Great Effort: You made strong progress throughout the course

Strengthen Your Prep: Reinforce your understanding before taking the exam

- **Refresh Concepts**: Review the presentation end-to-end
- **Practice Tests**: Attempt at least one practice test to understand exam patterns
- **Close Your Gaps**: Revisit areas where you did not do well in practice tests and re-watch videos as needed

in28minutes

WOW!

Congratulations

Let's Clap for You!

Congratulations: Congratulations on your patience and persistence

You Have Put Your Best Foot Forward toward getting certified

Final Step: Review the key concepts and complete your exam preparation

Good Luck!

Do Not Forget!

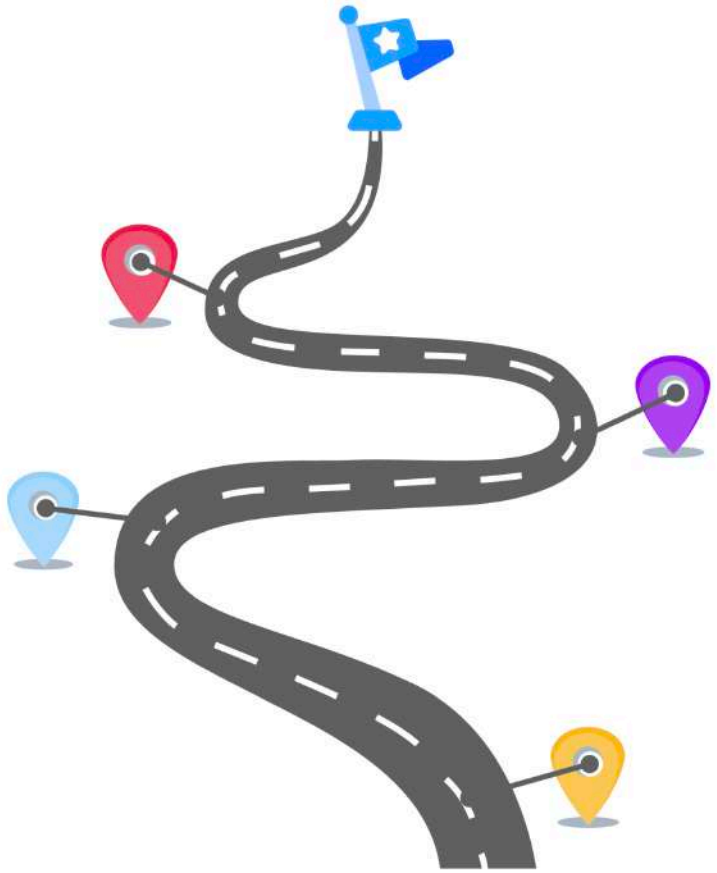
Recommend this Course to your friends

Leave a Review: Your feedback helps future learners

Your Success = Our Success

- Share your success story on LinkedIn (Tag: in28minutes and Ranga Karanam)
- Share your journey and lessons learned in Q&A to inspire others

Continue Your Learning Journey: Checkout our Roadmaps!



01 Generative AI Certifications

02 Cloud Certifications

03 Interview Guides

04 Programming

in28minutes

From Cloud to Generative AI - Learn, Build, and Evolve with in28minutes

in28minutes

Going Deeper with Generative AI

Section Overview - Going Deeper with Generative AI

Focus: What will we learn?

- Why do LLMs use tokens instead of words?
- How do LLMs predict the next token?
- How are Generative AI models trained?
- What is Self-Supervised Learning?
- What is Supervised Fine-Tuning?
- What is Reinforcement Learning from Human Feedback (RLHF)?

Foundation: Understand how modern Generative AI models work and learn

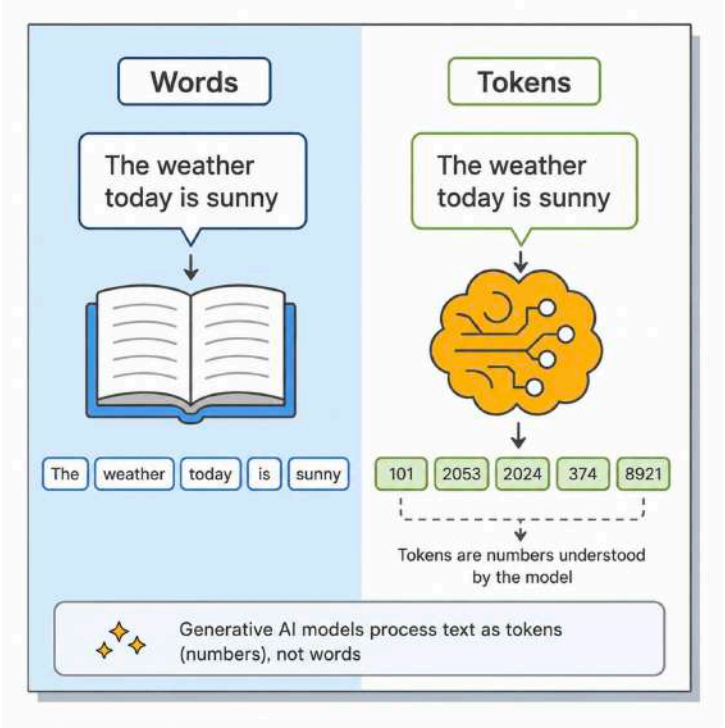
Large Language Models - Uses Tokens instead of Words

Token: A unit of text processed by the model

- Can be a word, part of a word, punctuation mark, or number

Why Tokens?

- **Efficient**: Whole words => large vocabulary
  - Tokens allow common words and word-parts to be reused
  - Examples: act, re + act, act + ing, view, re + view, view + ing
- **Handle Unseen Words**: Uncommon or new words can be split into smaller known tokens
  - Example: unfriendliness = un + friendli + ness
- **Support Diverse Text**: Tokens can represent different languages, code, numbers, symbols, and emojis



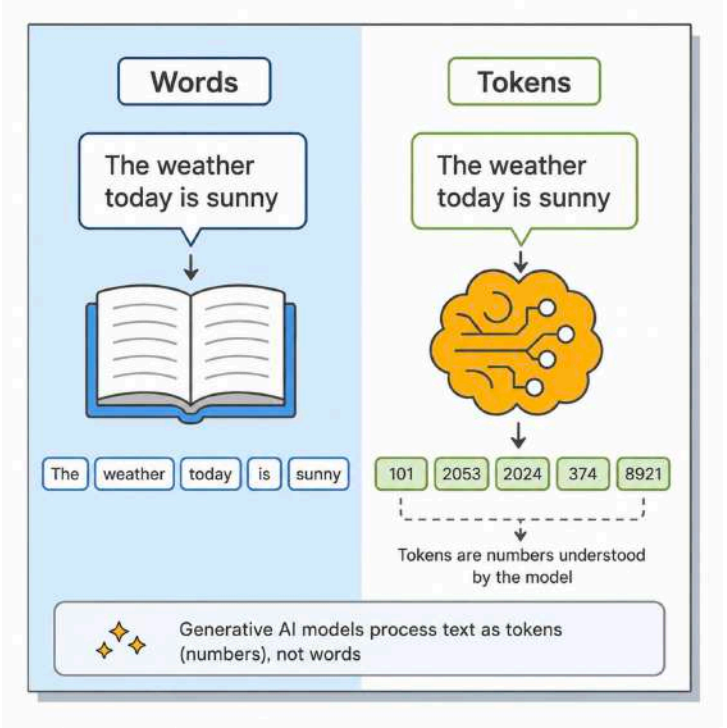
Key Step In Large Language Models - Decide Next Token [1/2]

Key Idea: Text generation is based on predicting the next token

During Training: Model learns probabilities for what could come next

- **Sequence**: "The cat sat on the"
- **Possible Next Tokens**: "mat": 0.4, "table": 0.2, "chair": 0.2, "moon": 0.1, ..

During Generation: Model might choose the highest probable token



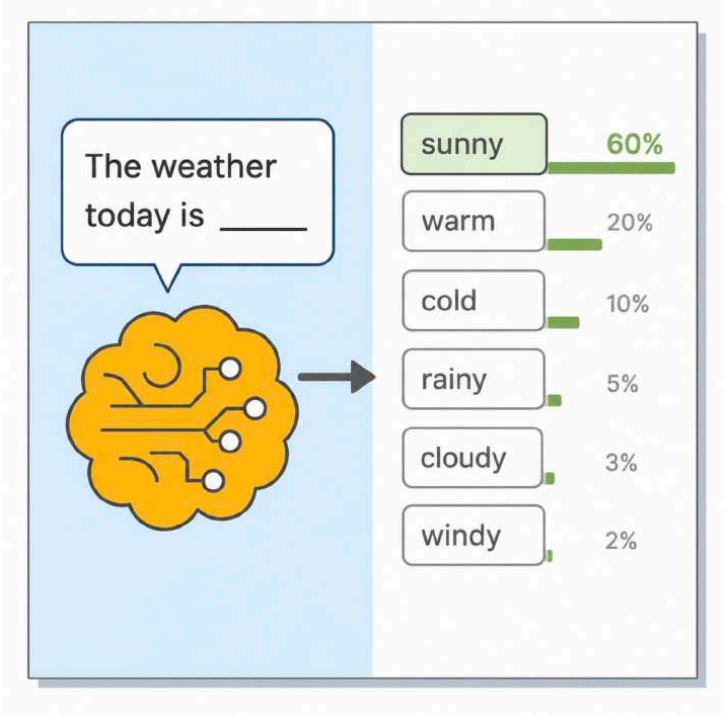
Key Step In Large Language Models - Decide Next Token [2/2]

During Generation: Model might choose the highest probable token

- The
- The weather
- The weather today
- The weather today is
- The weather today is sunny

Remember: The same approach is used to generate even large pieces of text

Temperature: Can influence the choice of next token



Generative AI - Uses Self Supervised Learning [1/2]

Self-Supervised Learning: Learn directly from training data

- No manual labels or annotations required

Training Data: "The capital of France is Paris"

- Use sequence: "The capital of France is"
- Try to predict the next word: "Paris"

Key Advantage: Can learn from huge volumes of text already available

- Does not need newly labeled training data

Generative AI - Uses Self Supervised Learning [2/2]

1 - Predict: Model tries to predict the next word based on preceding words

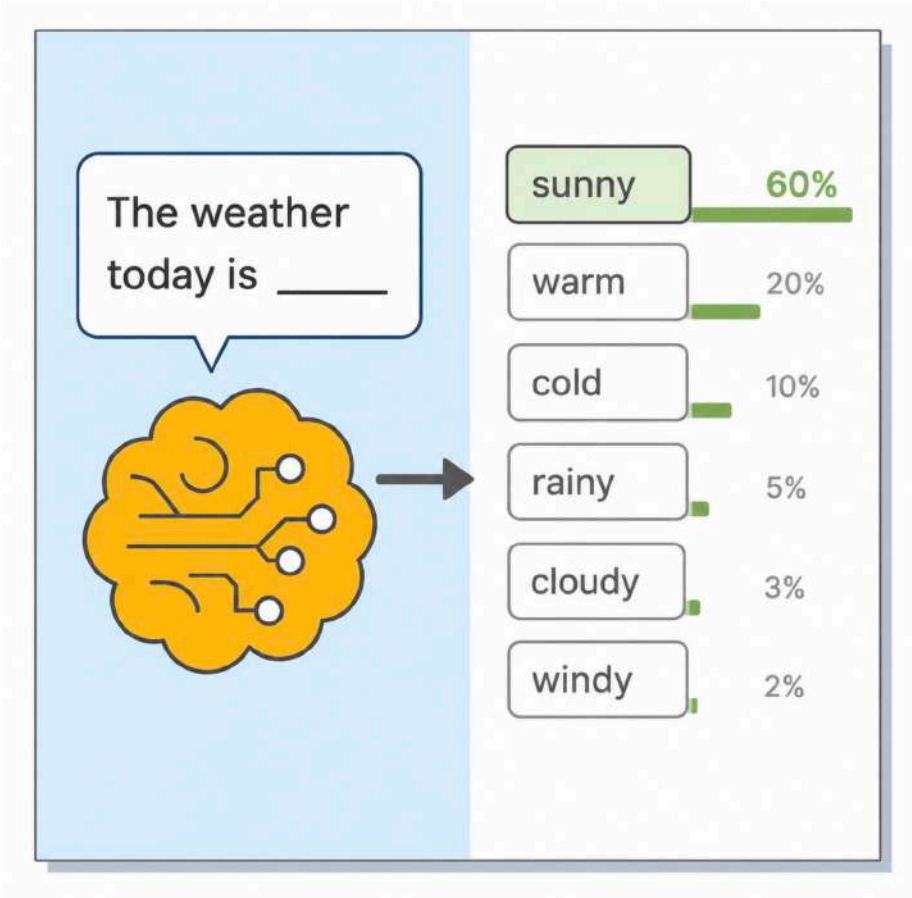
- Example: "The capital of France is"
- Model predicts the missing word

2 - Compare and Learn: Prediction is compared with the actual word

- Model learns from the error and adjusts its internal weights

3 - Repeat: Process is repeated across the entire training dataset

- Billions or trillions of examples can be used during training



Generative AI Text - Uses Supervised Fine-Tuning [1/2]

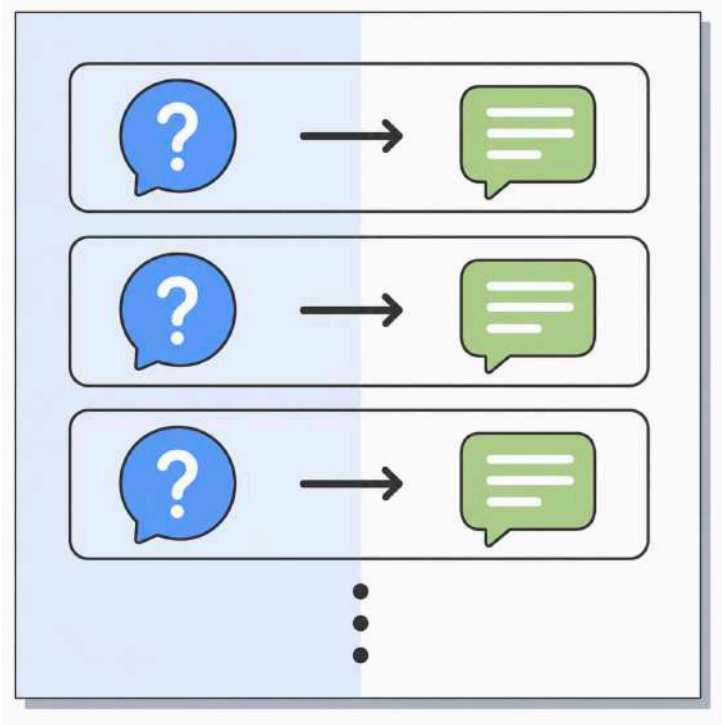
After Pre-Training: Model is good at predicting the next token

- **Example**: "My favorite sport is ...".
- Possible next tokens: basketball, soccer, cricket.

The Challenge: Next-token prediction does not guarantee Q&A behavior

- **Question**: How do we teach the model to answer questions?

Solution: Supervised Fine-Tuning (SFT)



Generative AI Text - Uses Supervised Fine-Tuning [2/2]

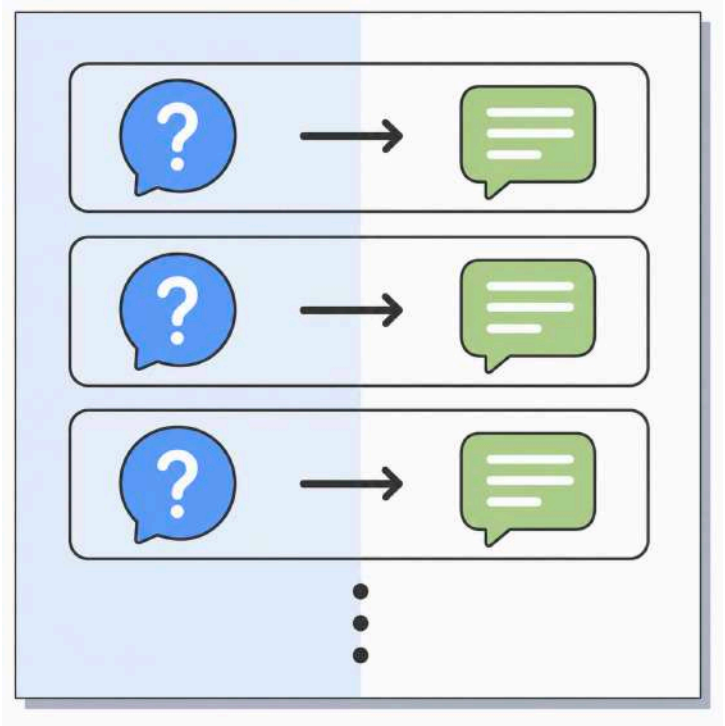
How It Works:

- Train using **Prompt + Ideal Response** examples
- Model learns patterns for answering questions and following instructions

The Result: Model also learns to respond in the expected way

- Answer questions
- Follow instructions
- Generate useful responses

Key Discovery: A relatively small, high-quality dataset can significantly improve behavior



Generative AI Text - Uses RLHF

The Challenge: Models do not inherently know which responses humans prefer

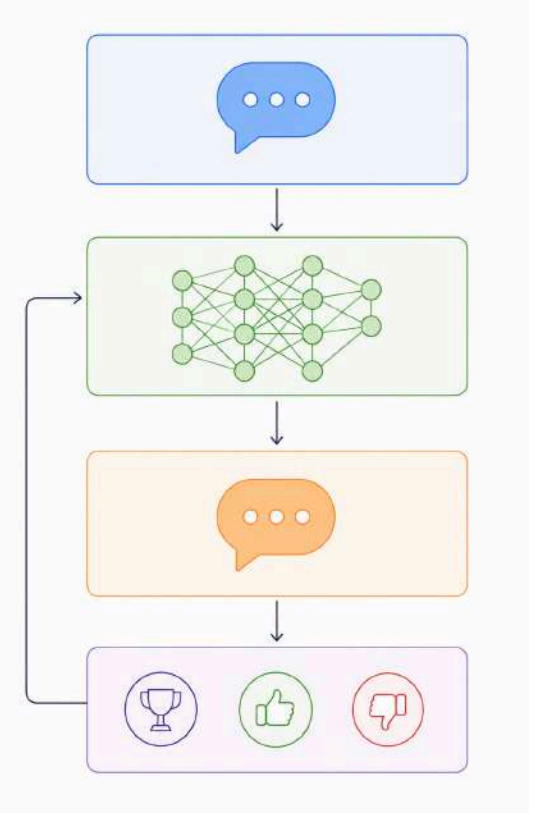
- Responses may be unhelpful, unsafe, biased, or inappropriate

Question: How can we better align model behavior with human preferences?

Solution: Reinforcement Learning from Human Feedback (RLHF)

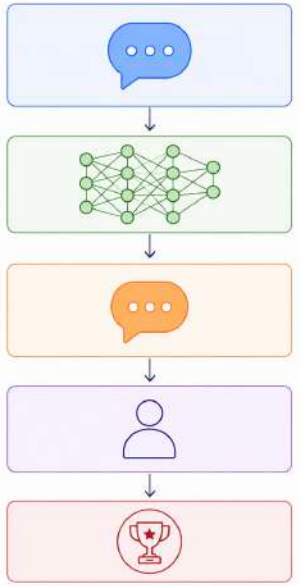
High-Level Steps:

- **Step 1**: Learn human preferences using a Reward Model
- **Step 2**: Tune the Generative AI model using the Reward Model



Generative AI Text - RLHF - Step 1

Goal: Train a Reward Model to recognize preferred responses



How?

- **1**: Generate multiple responses for the same prompt
- **2**: Human evaluators rank the responses
- **3**: Train the Reward Model using these preferences
- Repeat across many prompts and responses

Result: Reward Model learns to assign higher scores to responses humans tend to prefer

- These scores are called **rewards**

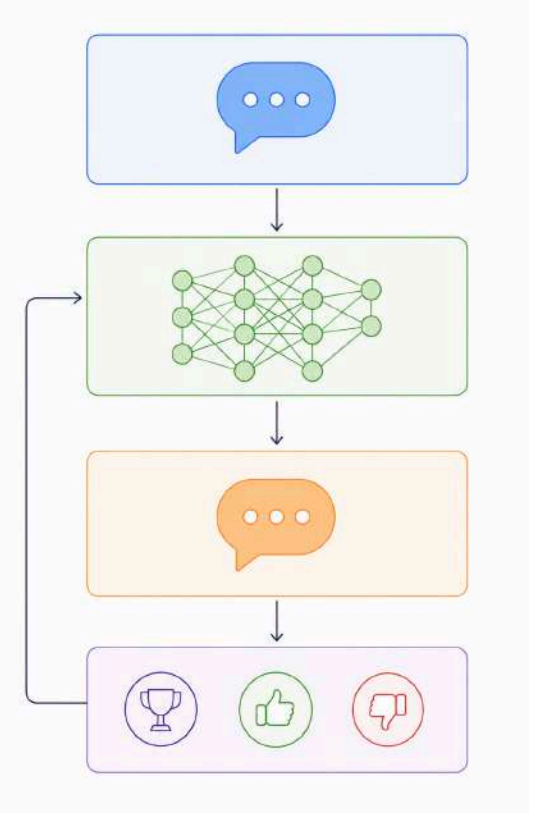
Generative AI Text - RLHF - Step 2

Goal: Tune the Generative AI model using the Reward Model

How?

- **1**: Generate a response for a prompt
- **2**: Reward Model evaluates the response
- **3**: Adjust the Generative AI model to increase expected reward
- Repeat across many training examples

Result: Model behavior becomes better aligned with learned human preferences



in28minutes

Journey to Generative AI

Embeddings to Transformer

Section Overview - Journey to Generative AI

Focus: What will we learn?

- What are embeddings and why do we need them?
- Why does language understanding need context?
- How do Recurrent Neural Networks process sequences?
- What problem does attention solve?
- Why do we need Multi-Head Attention?
- What is a Transformer?
- Why were Transformers a breakthrough?

Journey: Embeddings → RNNs → Attention → Transformers

What are Embeddings?

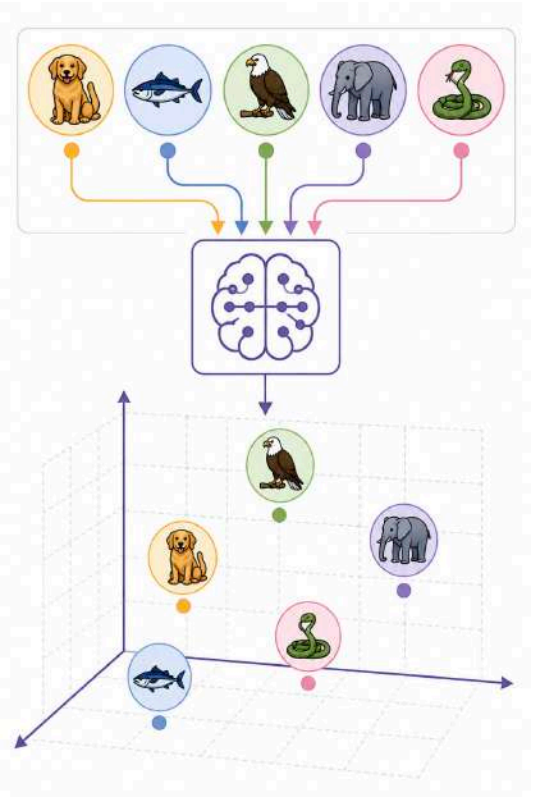
Simple Idea: Embeddings convert words or tokens into numbers that a neural network can work with

- Example: `"cat"` → `[0.12, -0.45, 0.78, ...]`

Vector: This list of numbers is a vector - each number represents a different dimension

- Simple dimension analogy for animals: [Habitat: water, land, or trees], [Diet: carnivore, herbivore, or omnivore], [Size: small, medium, or large], [Movement: flying, running, swimming, etc.]

Generative AI: Uses embeddings with hundreds or thousands of dimensions



What Can We Do with Embeddings?

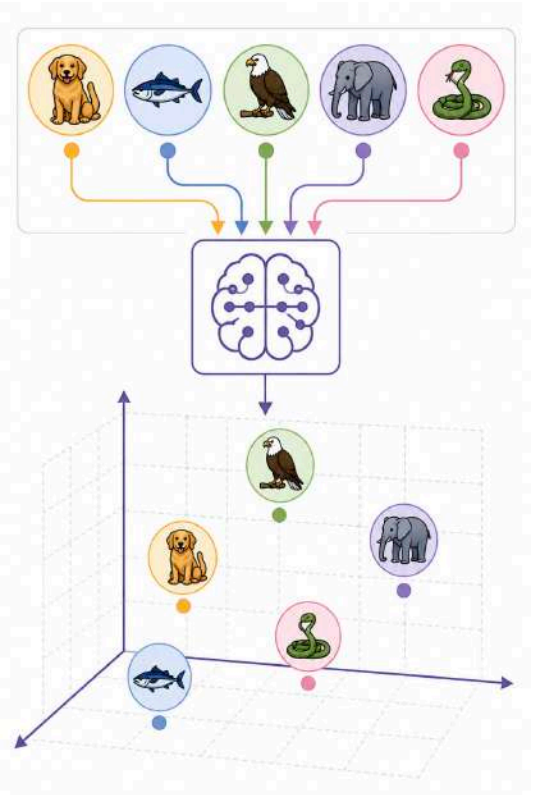
Similarity: Compare how closely related items are

- `"AWS"` may be closer to `"Azure"` than to `"football"`

Recommendations: Find similar products, users, movies, courses, or documents

Clustering: Automatically group similar items

Outlier Detection: Find items different from the rest



How are Embeddings Learned?

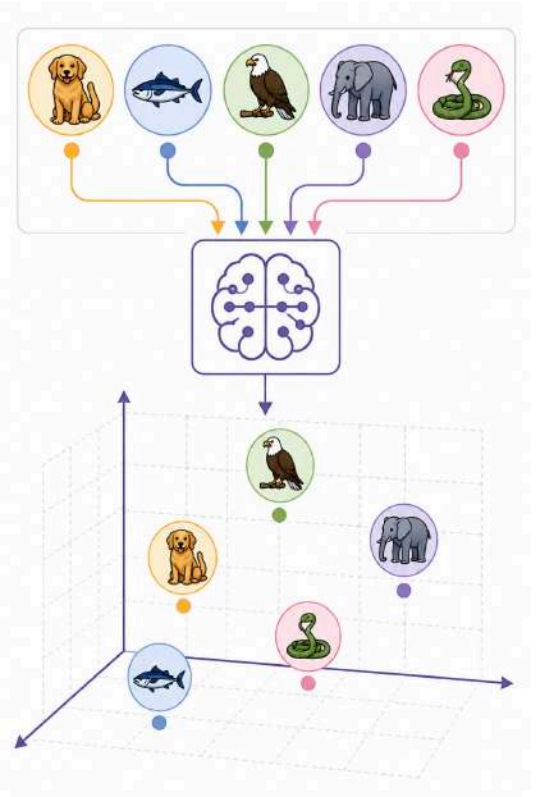
Step 1 - Start: Each token begins with an initial vector of numbers

- `"cat"` → `[0.12, -0.45, 0.78, ...]`

Step 2 - Train: The neural network processes huge amounts of training data

Step 3 - Learn: The numbers are repeatedly adjusted during training

Result: Tokens appearing in similar contexts develop useful relationships



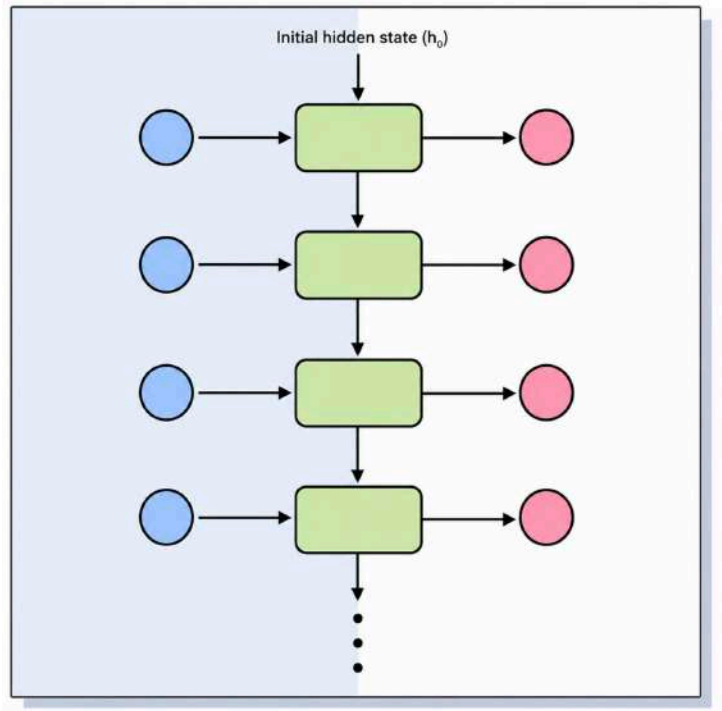
Language Needs Context

Individual Tokens: Embeddings provide useful information about each token

- **Language Understanding**: Also requires understanding relationships between tokens
- *Example*: `"The animal didn't cross the street because it was tired"` (What does `"it"` refer to?)
- **Challenge**: The model needs information from surrounding and earlier tokens

Early Solution: Recurrent Neural Networks

- Process tokens one after another
- Carry information from previous tokens forward



What is a Recurrent Neural Network?

Goal: Process sequential data such as text

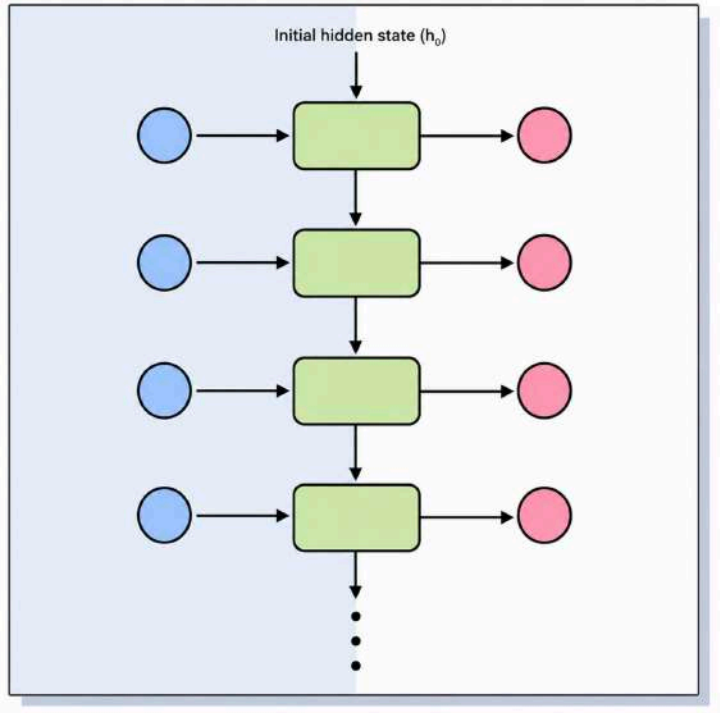
Key Idea: Process one token at a time

- First token → Second token → Third token → ...

Memory: Information from earlier tokens is carried forward

Example:

- "The cat is very ..."
- Information about "cat" is carried forward while processing later words



How Does an RNN Work?

Step 1 - Read Token: Read the current token

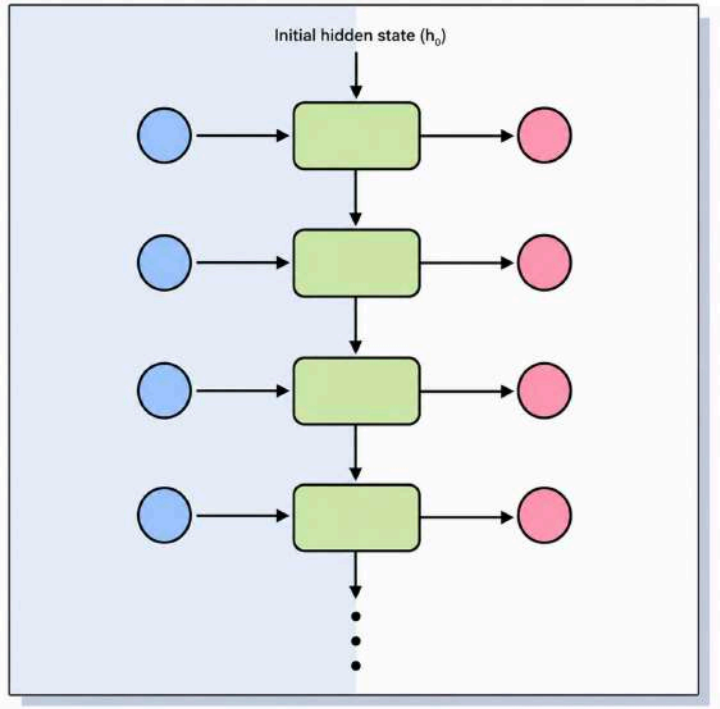
Step 2 - Read Previous Memory: Use information carried from earlier tokens

Step 3 - Combine: Combine the current token with previous information

Step 4 - Update Memory: Create a new hidden state

Step 5 - Move Forward: Pass the hidden state to the next token

Repeat: Continue till all tokens are processed

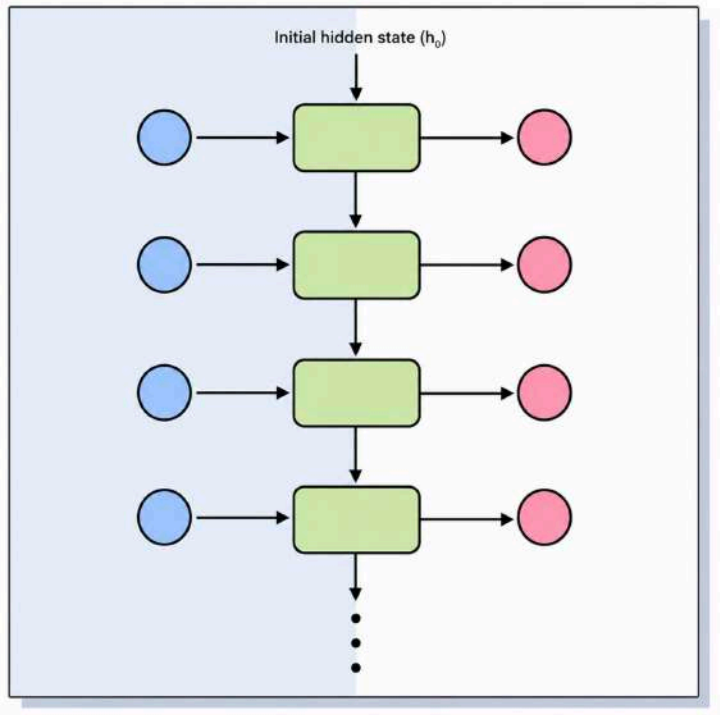


Hidden State - Memory of an RNN

Hidden State: Internal memory carried from one step to the next

- **Purpose**: Store useful information about previously processed tokens
- **Example**:
  - "Ranga lives in Hyderabad. He teaches..."
  - When processing "He", earlier information about "Ranga" can help

Limitation: Difficulty preserving important information across very long sequences



What is the Problem with RNNs?

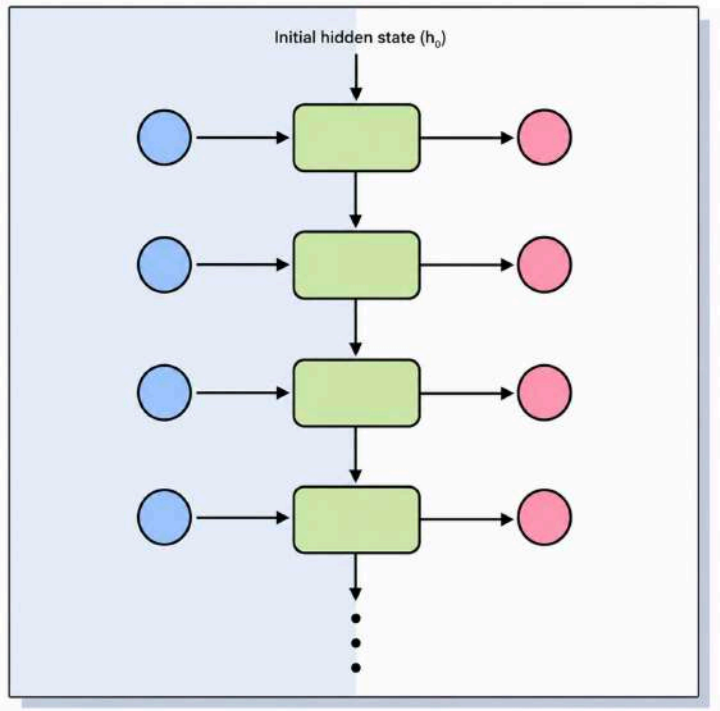
Long-Term Memory: Information from much earlier tokens can be difficult to preserve

- **Long Sequences**: The farther information has to travel, the harder it can be to use effectively

Sequential Processing: Tokens must be processed one after another

- Token 2 waits for Token 1, Token 3 waits for Token 2
- **Training Challenge**: Sequential processing limits parallelism

Question: Can a token directly look at the most relevant tokens?



What is Attention?

Goal: Determine which other tokens are important for understanding current token

For Each Token:

- Compare it with other tokens
- Calculate attention scores
- Give more weight to relevant tokens

Result: Each token gets a context-aware representation

Attention - Simple Example

Sentence: "The cat sat on the mat"

Focus on "sat":

- "cat" → High attention
- "the" → Lower attention
- "mat" → Some attention
- **Why?**: "cat" tells us who performed the action

Focus on "on":

- "mat" → High attention

Attention Changed the Game

Attention: Allows a token to directly focus on other relevant tokens

Example: "The animal didn't cross the street because it was tired"

- **Understanding "it"**: "it" can pay strong attention to "animal"
  - Less attention can be given to less relevant tokens

Benefit: Relevant relationships can be identified even when tokens are far apart

RNN vs Attention

RNN: Information passes step by step through the sequence

- Token 1 → Token 2 → Token 3 → .. → Token 10
- **RNN Challenge**: Long-distance information can be difficult to preserve

Attention: Tokens can directly focus on other relevant tokens

- Token 10 can directly focus on Token 1
- **Attention Benefit**: Direct connections between related tokens
- **Result**: Better understanding of relationships across a sequence

Why Multi-Head Attention?

Challenge: A sentence can contain many different relationships between tokens

- **Single Attention Head**: May focus on only some of these relationships

Solution: Use multiple attention heads

- **Each Head**: Learns to focus on different relationships

Illustrative Example: *"The cat sat on the mat"*

- **Head 1 - Proximity**: "sat" ↔ "on"
- **Head 2 - Subject and Action**: "cat" ↔ "sat"
- **Head 3 - Preposition and Object**: "on" ↔ "mat"
- ...

Multi-Head Attention – Why Multiple Heads?

Different Perspectives: Each attention head processes the same tokens from a different perspective

Different Relationships: Heads can learn different patterns between tokens

Parallel Attention: Multiple relationships can be captured at the same time

Combined Output: Information from all heads is combined

Result: Richer contextual representation of each token

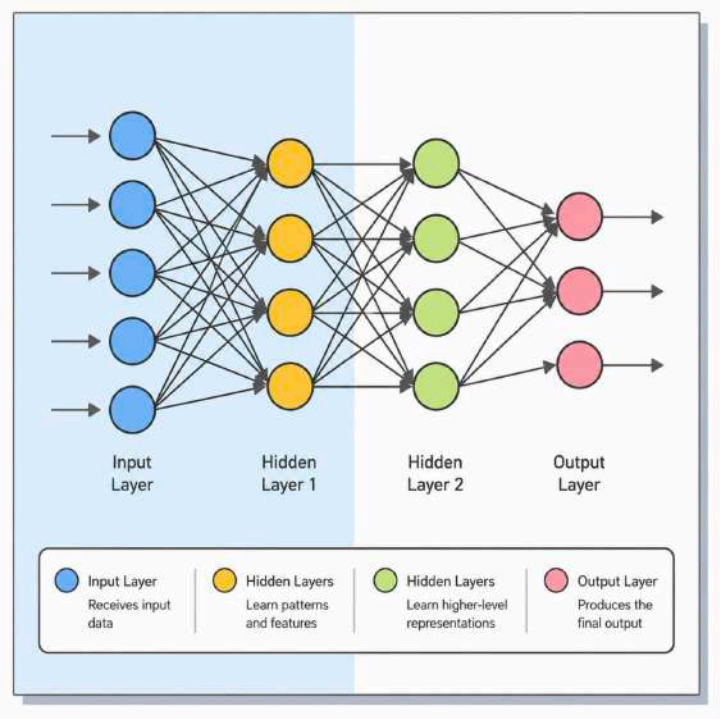
From Attention to Transformers

RNNs: Process tokens one after another and carry information forward

Attention: Allows tokens to directly focus on other relevant tokens

- **Key Question**: What if we build a neural network mainly around attention?

Transformer: Use attention as a core mechanism

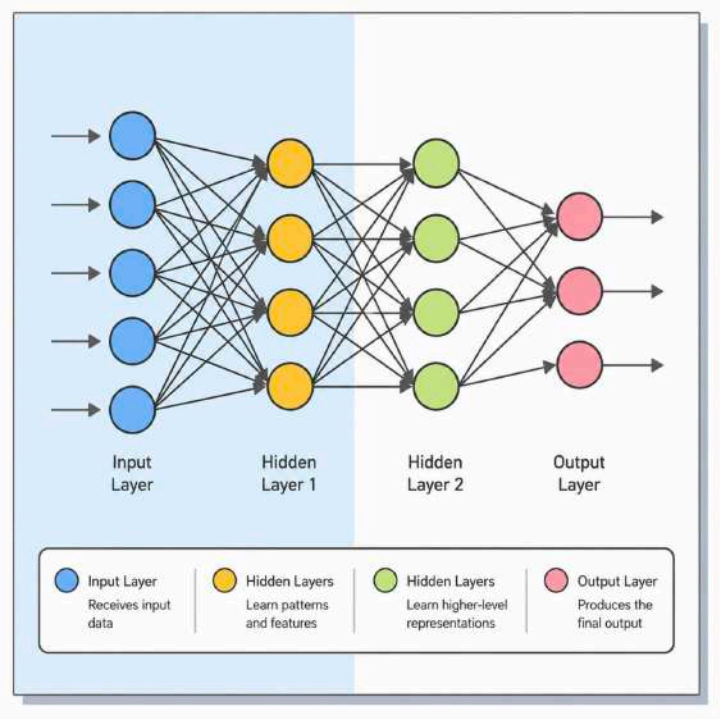


What is a Transformer?

Transformer: Neural network architecture designed to process sequences using attention

Core Building Blocks:

- **Embeddings**: Represent tokens as vectors
- **Multi-Head Attention**: Understand relationships between tokens
- **Neural Network Layers**: Process and refine the representations



Why Were Transformers a Breakthrough?

Better Context: Attention allows each token to directly consider other relevant tokens

Parallel Processing: Many tokens can be processed in parallel during training

- **Scalable**: Works efficiently with massive datasets and compute

Flexible: Can be used for text, images, audio, and multimodal data

Foundation: Became the architecture behind many modern Generative AI models